CRISP-DA

Calibrated Reliability-Informed Structured Pseudo-labeling for Domain Adaptation

Đặc tả chi tiết kiến trúc end-to-end, cơ chế huấn luyện và toàn bộ hàm mục tiêu

cho Unsupervised Domain Adaptation in Semantic Segmentation (UDASS)

Tóm tắt. Tài liệu này mô tả đầy đủ một thiết kế nghiên cứu CRISP-DA cho bài toán Unsupervised Domain Adaptation in Semantic Segmentation. Hệ thống sử dụng teacher–student self-training, Class-Conditional Multi-Prototype Distribution Bank, Multi-Evidence Trust Estimator, Four-State Pseudo-Label Policy và Balanced Progressive Coverage Curriculum để biến raw pseudo-label thành supervision có cấu trúc. Tài liệu trình bày input/output và tensor shape của từng module, luồng dữ liệu end-to-end, cơ chế cập nhật EMA Teacher và Distribution Bank, các objective/loss, lịch huấn luyện theo giai đoạn và quy trình inference. Các công thức và hyperparameter trong tài liệu là một **research design proposal**; chúng cần được kiểm chứng bằng ablation và thực nghiệm, không nên được diễn đạt như một phương pháp đã được công bố nguyên trạng.

Phiên bản kỹ thuật mở rộng – 2026

Mục lục

1 Tóm tắt điều hành **5**

2 Phạm vi, giả định và bài toán **5**
2.1 Setting UDA chuẩn . . . 5
2.2 Giả định phiên bản cơ sở . . . 5
2.3 Mục tiêu thiết kế . . . 5

3 Động lực thiết kế và đóng góp dự kiến **6**
3.1 Vì sao confidence-only self-training là chưa đủ? . . . 6
3.2 Vì sao cần structured pseudo-label actions? . . . 6
3.3 Vì sao cần multi-prototype class distribution? . . . 6
3.4 Vì sao cần class-balanced progressive coverage? . . . 6
3.5 Các đóng góp kỹ thuật được kỳ vọng . . . 6

4 Kiến trúc tổng thể end-to-end **7**
4.1 Bảy thành phần chính . . . 7
4.2 Luồng tổng quát từ input đến output . . . 7
4.3 Computational graph và nguyên tắc tách luồng gradient . . . 8
4.4 Hai resolution cần phân biệt trong toàn pipeline . . . 8

5 Student Segmentation Network **8**
5.1 Vai trò . . . 8
5.2 Input và output . . . 8
5.3 Chi tiết MiT-B5 encoder . . . 8
5.4 Dense projection và fusion . . . 9
5.5 Segmentation head . . . 9
5.6 Source supervised learning . . . 9

6 EMA Teacher Network **9**
6.1 Kiến trúc và cập nhật . . . 9
6.2 Input . . . 9
6.3 Output . . . 9
6.4 Teacher–student view construction và geometric alignment . . . 10
6.5 Teacher outputs được dùng ở đâu? . . . 10

7 Boundary Head **10**
7.1 Motivation . . . 10
7.2 Layer design . . . 10
7.3 Source boundary ground truth . . . 11
7.4 Target spatial role . . . 11

8 Class-Conditional Multi-Prototype Distribution Bank **11**
8.1 Vai trò . . . 11
8.2 Representation . . . 11
8.3 Source bootstrap . . . 11
8.4 Distribution compatibility . . . 11
8.5 Prototype posterior trên toàn bộ class . . . 12
8.6 Online update: source bootstrap và reliable-target adaptation . . . 12
8.7 Mode assignment trong online update . . . 13
8.8 Reliable-target update và contamination control . . . 14
8.9 Stabilization của mixture weights và rare modes . . . 14
8.10 Outputs chính xác của Bank . . . 14

9 Multi-Evidence Trust Estimator **14**
9.1 Mục tiêu và input/output . . . 14
9.2 Teacher confidence q_i: predictive certainty . . . 14
9.3 Augmentation consistency a_i: perturbation stability . . . 15
9.4 Distribution compatibility d_i: semantic-feature agreement . . . 15
9.5 Temporal stability s_i: consistency qua thời gian . . . 15

9.6 Boundary evidence b_i: spatial difficulty . . . 15
9.7 Class conditioning . . . 15
9.8 Kiến trúc estimator . . . 15
9.9 Huấn luyện MTE bằng source correctness calibration . . . 15
9.10 Lý do chọn năm evidence và tính bổ sung của chúng . . . 17
9.11 Pixel-wise architecture và lý do dùng pointwise Conv . . . 17
9.12 Calibration target, class imbalance và validation của Trust Estimator . . . 17

10 Four-State Pseudo-Label Policy **17**
10.1 Vai trò, input và output . . . 17
10.2 ACCEPT: giữ Teacher hard label . . . 17
10.3 REVISE: sửa hard pseudo-label bằng alternative evidence . . . 17
10.4 SOFT SUPERVISE: giữ uncertainty thay vì ép integer class . . . 18
10.5 ABSTAIN: không dùng pseudo-supervision tại iteration hiện tại . . . 18
10.6 Thứ tự ra quyết định và các điều kiện xung đột . . . 18
10.7 Supervision masks và weighting . . . 19
10.8 Liên hệ giữa policy và Bank update . . . 19

11 Balanced Progressive Coverage Curriculum **19**
11.1 Mục tiêu và input/output . . . 19
11.2 Class-wise coverage và reliability . . . 19
11.3 Progressive target coverage . . . 19
11.4 Class-balanced threshold controller . . . 20
11.5 Quantile controller: lựa chọn thay thế dễ triển khai . . . 20
11.6 Feedback loop giữa BPCC và Four-State Policy . . . 20
11.7 Statistics nên log để kiểm tra curriculum . . . 21

12 Target strong-view training **21**
12.1 Strong augmentation . . . 21
12.2 Student forward . . . 21

13 Thiết kế các hàm loss **21**
13.1 Source segmentation loss . . . 21
13.2 Hard target loss . . . 21
13.3 Soft target loss . . . 21
13.4 Consistency loss . . . 21
13.5 Target boundary loss . . . 21
13.6 Prototype loss . . . 22
13.7 Trust calibration loss . . . 22
13.8 Total objective . . . 22
13.9 Masking, normalization và gradient routing của từng loss . . . 22
13.10 Weighted total objective và lịch bật loss . . . 22

14 Quy trình training đầy đủ **23**
14.1 Stage 0: preprocessing . . . 23
14.2 Stage 1: source bootstrap . . . 23
14.3 Stage 2: conservative adaptation . . . 23
14.4 Stage 3: full CRISP-DA . . . 23
14.5 Một iteration đầy đủ . . . 23
14.6 Thứ tự update trong một iteration để tránh leakage . . . 24
14.7 Warm-up và chuyển pha . . . 24
14.8 Optimizer và state update không đồng tốc . . . 24

15 Pseudo-code tham khảo **24**

16 Kiểm tra tính đúng đắn của luồng và các invariant triển khai **25**

17 Phân tích độ phức tạp và chi phí bộ nhớ **25**

18 Tensor I/O specification **26**

19 Cấu hình huấn luyện khuyến nghị để bắt đầu **26**

20 Inference **26**

21 Hướng dẫn vẽ sơ đồ kiến trúc trong diagrams.net **27**
21.1 Bố cục đề xuất . . . 27
21.2 Mũi tên bắt buộc . . . 27
21.3 Không nên vẽ . . . 27

22 Ablation bắt buộc **28**

23 Evaluation protocol **28**

24 Research hypotheses **28**

25 Giới hạn và rủi ro của kiến trúc đề xuất **29**
25.1 Prototype poisoning . . . 29
25.2 Source-corruption calibration gap . . . 29
25.3 Temporal memory cost . . . 29
25.4 Threshold/controller sensitivity . . . 29
25.5 Closed-set limitation . . . 29

26 Phiên bản tối giản để triển khai V1 **29**

27 Checklist triển khai **29**

28 Kết luận **30**

A Bảng cấu hình module chi tiết **31**

B Ký hiệu **31**

Danh sách hình vẽ

1 Kiến trúc end-to-end của CRISP-DA. Source branch cung cấp supervision sạch và feature statistics; target weak views được xử lý bởi EMA Teacher để tạo raw pseudo-label và evidence; Distribution Bank và Trust Estimator đánh giá độ tin cậy; Four-State Policy tạo refined supervision cho strong-view Student. Teacher và Distribution Bank được cập nhật ngoài gradient, trong khi Student/Boundary/Trust modules được tối ưu bằng backpropagation. 7
2 Minh họa Student/EMA-Teacher backbone và Boundary Head. Source branch tạo dense semantic feature, segmentation output và boundary output; target được tạo thành nhiều weak views và đưa qua EMA Teacher. Hình mang tính sơ đồ minh họa cho luồng dữ liệu, không quy định duy nhất một backbone cụ thể. 10
3 Minh họa xây dựng Class-Conditional Multi-Prototype Distribution Bank từ dense feature: mỗi semantic class được mô hình hóa bởi nhiều mode với π_{ck}, μ_{ck} và Σ_{ck}. 13
4 Minh họa target-query: target dense feature được đối chiếu với các class-wise modes để tạo Distribution Compatibility d_i và Prototype Class Distribution p_i^{proto}. 13
5 Multi-Evidence Trust Estimator: năm evidence bổ sung cho nhau được fuse thành reliability score thay vì dùng max-softmax confidence như một quyết định nhị phân. 16
6 Minh họa trực quan các evidence maps của MTE. Đây là hình minh họa khái niệm cho cách biểu diễn signal theo không gian, không phải kết quả định lượng của một checkpoint CRISP-DA đã được huấn luyện. . . . 16
7 Four-State Pseudo-Label Policy. ACCEPT và REVISE tạo hard targets; SOFT giữ distribution uncertainty; ABSTAIN tạm thời không tạo pseudo-label supervision. 18
8 BPCC điều khiển threshold theo class và training stage. Module sử dụng statistics, không có trainable parameters. 20

Danh sách bảng

1 Cấu hình canonical của MiT-B5 trong CRISP-DA. 8
2 Ý nghĩa của các loss trong CRISP-DA. 22
3 Input/output của từng thành phần. 26
4 Module-level implementation specification. 31

1 Tóm tắt điều hành

CRISP-DA là một kiến trúc UDASS theo khung **teacher–student self-training**, nhưng thay thế cách chọn pseudo-label dựa trên một confidence threshold duy nhất bằng một cơ chế **multi-evidence calibrated trust**. Với mỗi target pixel hoặc target region, hệ thống tổng hợp nhiều nguồn bằng chứng: confidence của EMA teacher, tính nhất quán dưới augmentation, độ phù hợp với phân phối feature theo class, độ ổn định theo thời gian và vai trò không gian interior/boundary. Một Trust Estimator học cách chuyển các evidence này thành reliability score. Sau đó, Four-State Pseudo-Label Policy quyết định một trong bốn hành động: **ACCEPT**, **REVISE**, **SOFT SUPERVISE**, hoặc **ABSTAIN**. Cuối cùng, Balanced Progressive Coverage Curriculum điều chỉnh mức độ mở rộng pseudo-label theo từng class trong suốt quá trình training.

Kiến trúc được thiết kế để xử lý bốn failure mode liên hệ chặt chẽ trong self-training: (i) confidence không đồng nghĩa correctness; (ii) confirmation bias của EMA teacher; (iii) class imbalance và head-class dominance; (iv) boundary/hard-region thường bị bỏ qua vì confidence thấp. Ý tưởng tham khảo một số cơ chế đã xuất hiện riêng lẻ trong các hướng gần đây như EMA/self-training [5, 8], prototype-based correction [7], class-balanced curriculum [6], và multi-prototype distribution modeling [9], nhưng CRISP-DA tổ chức chúng quanh một giả thuyết trung tâm khác: *pseudo-label reliability phải được ước lượng bằng nhiều bằng chứng, rồi supervision phải được sử dụng theo nhiều mức thay vì Keep/Ignore nhị phân*.

Core hypothesis

Thay vì xem pseudo-label reliability là một scalar lấy trực tiếp từ softmax confidence, CRISP-DA mô hình hóa

$$R_i \approx P(\tilde{y}_i = y_i \mid \text{class}, \text{spatial role}, \text{prediction evidence}, \text{distribution evidence}, \text{temporal evidence}),$$

rồi ánh xạ reliability thành bốn chế độ supervision khác nhau.

2 Phạm vi, giả định và bài toán

2.1 Setting UDA chuẩn

Ta có source domain có nhãn

$$\mathcal{D}_s = \{(x_s^n, y_s^n)\}_{n=1}^{N_s},$$

và target domain không nhãn

$$\mathcal{D}_t = \{x_t^m\}_{m=1}^{N_t}.$$

Ảnh RGB được ký hiệu

$$x \in \mathbb{R}^{B\times 3\times H\times W},$$

segmentation label của source

$$y_s \in \{0, \dots, C-1\}^{B\times H\times W}.$$

Trong Cityscapes-like protocol, thường $C = 19$.

Mục tiêu là học một segmentation network f_θ có khả năng dự đoán semantic map trên target domain mà không sử dụng y_t trong training.

2.2 Giả định phiên bản cơ sở

Phiên bản đầu tiên của CRISP-DA giả định closed-set UDA:

$$\mathcal{C}_s = \mathcal{C}_t.$$

Không xử lý unknown class hay taxonomy shift trong phiên bản này. Kiến trúc cũng giả định target dataset có thể được truy cập lặp lại trong training để duy trì temporal memory. Với online TTA, temporal module cần thiết kế lại.

2.3 Mục tiêu thiết kế

CRISP-DA ưu tiên năm mục tiêu:

1. Giảm phụ thuộc vào raw softmax confidence.
2. Sửa được một phần pseudo-label sai thay vì chỉ lọc bỏ.
3. Không ép hard label cho các pixel còn mơ hồ.
4. Bảo vệ tail classes và boundary/hard regions khỏi việc bị loại bỏ kéo dài.
5. Giữ inference gần như bằng baseline segmentation network.

3 Động lực thiết kế và đóng góp dự kiến

3.1 Vì sao confidence-only self-training là chưa đủ?

Trong self-training cho UDASS, EMA Teacher thường sinh pseudo-label bằng $\tilde{y}_i^T = \arg\max_c p_{i,c}^T$ và một confidence score $q_i = \max_c p_{i,c}^T$. Cách làm này đơn giản nhưng có một điểm yếu cơ bản: q_i chỉ đo mức độ tập trung của phân phối dự đoán hiện tại, không trực tiếp đo xác suất pseudo-label là đúng dưới domain shift. Một target pixel có thể đạt $q_i \approx 0.95$ nhưng feature của nó nằm xa phân phối source của class được dự đoán; ngược lại, một pixel ở boundary có q_i thấp hơn nhưng vẫn chứa tín hiệu đúng và hữu ích. Do đó, dùng một global threshold τ để quyết định Keep/Ignore có thể đồng thời tạo ra hai lỗi: giữ lại các pseudo-label "confidently wrong" và loại bỏ quá nhiều hard-but-informative pixels.

CRISP-DA xuất phát từ giả thuyết rằng reliability nên được phân rã thành nhiều bằng chứng có ý nghĩa khác nhau. Predictive confidence trả lời "Teacher tự tin đến đâu?"; augmentation consistency kiểm tra prediction có bền vững trước perturbation hay không; distribution compatibility kiểm tra target feature có phù hợp với class-conditional semantic distribution hay không; temporal stability kiểm tra prediction có ổn định qua training; và boundary evidence mô tả mức độ khó về không gian. Mục tiêu không phải chứng minh các evidence hoàn toàn độc lập thống kê, mà là cung cấp các góc nhìn bổ sung để giảm phụ thuộc vào một scalar confidence duy nhất.

3.2 Vì sao cần structured pseudo-label actions?

Một hạn chế khác của nhiều pipeline là pseudo-label cuối cùng chỉ có hai trạng thái: dùng hoặc bỏ. Tuy nhiên, uncertainty trong segmentation không đồng nhất. Có pixel đủ đáng tin để giữ nguyên; có pixel mà Teacher đáng nghi nhưng prototype distribution hỗ trợ rõ một class khác; có pixel chưa đủ chắc để hard-assign nhưng phân phối xác suất vẫn giàu thông tin; và có pixel quá mơ hồ nên tốt nhất tạm thời không học. CRISP-DA vì thế mô hình hóa pseudo-label refinement như một routing problem gồm bốn hành động: ACCEPT, REVISE, SOFT SUPERVISE và ABSTAIN. Điểm thiết kế cốt lõi là *loại supervision* được quyết định sau khi reliability đã được hiệu chỉnh, thay vì biến reliability thành một mask nhị phân ngay lập tức.

3.3 Vì sao cần multi-prototype class distribution?

Một semantic class không nhất thiết là một cụm cầu (spherical cluster) quanh một centroid duy nhất. Ví dụ feature của class `car` có thể khác nhau đáng kể giữa xe gần, xe xa, xe bị che khuất, xe trong bóng tối và xe ở điều kiện mưa; class `road` có thể chứa asphalt khô, asphalt ướt, vùng phản chiếu và các texture khác. Một prototype duy nhất có thể làm trung bình hóa các mode này và cho khoảng cách không phản ánh đúng semantic compatibility. Vì vậy CRISP-DA dùng K_c mode cho mỗi class và mô tả mode bằng $(\pi_{ck}, \mu_{ck}, \Sigma_{ck})$. Bank đóng vai trò một semantic memory: source GT cung cấp reference sạch ban đầu, còn reliable target features dần dịch chuyển bank về target domain mà không cho noisy pixels làm nhiễm bộ nhớ quá sớm.

3.4 Vì sao cần class-balanced progressive coverage?

Pseudo-label quality và learning speed khác nhau rõ giữa các class. Nếu mọi class dùng cùng một threshold, các class dễ/chiếm diện tích lớn như `road`, `building`, `sky` thường đạt confidence cao sớm và tạo phần lớn supervision. Các class nhỏ hoặc khó như `rider`, `pole`, `traffic sign` có ít pseudo-label hơn, vì thế cải thiện chậm hơn và càng khó vượt threshold trong các epoch sau. Đây là một feedback loop bất lợi. BPCC được thiết kế để theo dõi coverage và reliability theo class, ưu tiên precision ở đầu training, rồi mở rộng coverage dần khi class đó thực sự ổn định.

3.5 Các đóng góp kỹ thuật được kỳ vọng

Ở mức thiết kế nghiên cứu, CRISP-DA tập trung vào bốn đóng góp dự kiến. Thứ nhất là **multi-evidence reliability calibration** ở mức pixel, trong đó reliability r_i được ước lượng từ nhiều signal thay vì lấy trực tiếp từ max-softmax. Thứ hai là **four-state supervision routing**, biến reliability thành bốn cách sử dụng pseudo-label khác nhau thay vì Keep/Ignore. Thứ ba là **reliability-gated class-conditional distribution adaptation**: multi-prototype bank được bootstrap từ source và chỉ nhận target features sau khi vượt qua policy tin cậy, nhằm cân bằng giữa target adaptation và contamination control. Thứ tư là **balanced progressive coverage**, cho phép threshold thay đổi theo class và theo thời gian training. Các thành phần nền tảng như EMA teacher, prototype/GMM modeling hay soft targets đã có prior art; phần cần được kiểm chứng thực nghiệm là hiệu quả của cách tổ chức chúng thành một pipeline thống nhất quanh reliability-aware decision making.

Thông điệp phương pháp

CRISP-DA không đặt câu hỏi duy nhất "pseudo-label này có đủ confidence để dùng hay không?". Pipeline tách thành ba câu hỏi tuần tự: (1) pseudo-label đáng tin đến đâu? (2) nếu chưa hoàn toàn tin, evidence nào cho biết nên giữ, sửa, dùng mềm hay bỏ qua? (3) ở giai đoạn hiện tại, class này nên được phép mở rộng coverage đến mức nào?

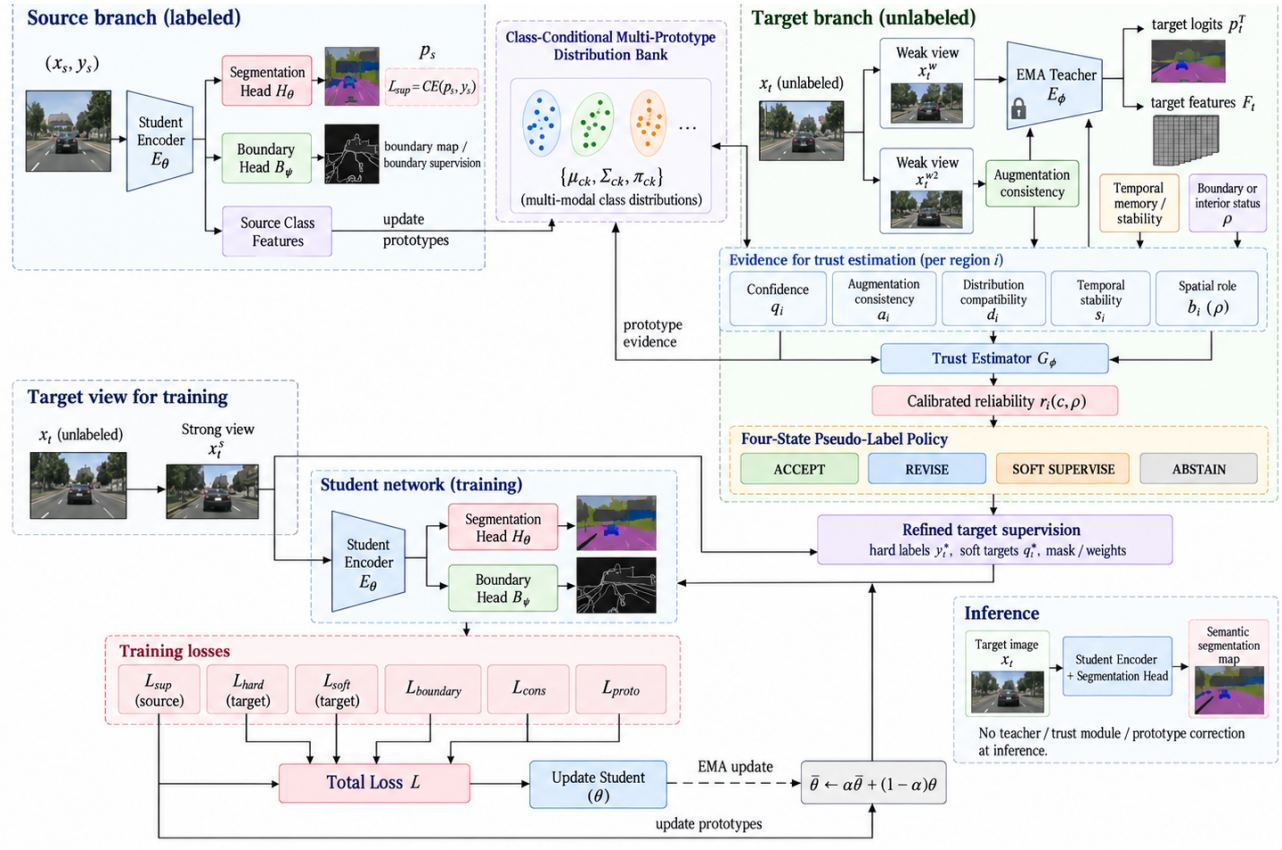
4 Kiến trúc tổng thể end-to-end

4.1 Bảy thành phần chính

CRISP-DA gồm bảy thành phần:

1. Student Segmentation Network.
2. EMA Teacher Network.
3. Boundary Head.
4. Class-Conditional Multi-Prototype Distribution Bank.
5. Multi-Evidence Trust Estimator.
6. Four-State Pseudo-Label Policy.
7. Balanced Progressive Coverage Curriculum.

4.2 Luồng tổng quát từ input đến output



Hình 1: Kiến trúc end-to-end của CRISP-DA. Source branch cung cấp supervision sạch và feature statistics; target weak views được xử lý bởi EMA Teacher để tạo raw pseudo-label và evidence; Distribution Bank và Trust Estimator đánh giá độ tin cậy; Four-State Policy tạo refined supervision cho strong-view Student. Teacher và Distribution Bank được cập nhật ngoài gradient, trong khi Student/Boundary/Trust modules được tối ưu bằng backpropagation.

Ở mỗi iteration, source branch cung cấp supervised segmentation signal, boundary supervision và clean class features để xây bank. Target branch sinh weak views cho EMA teacher, từ đó lấy logits, raw pseudo-label và dense features. Các evidence được tổng hợp vào Trust Estimator; Four-State Policy biến reliability thành supervision phù hợp; strong target view được đưa vào student để thực hiện adaptation. Student được cập nhật bằng gradient, teacher được cập nhật bằng EMA, prototype bank và temporal memory được cập nhật ngoài gradient.

4.3 Computational graph và nguyên tắc tách luồng gradient

Một điểm quan trọng của kiến trúc là không phải mọi module đều được cập nhật bằng backpropagation. Student segmentation network, Boundary Head và Trust Estimator có trainable parameters và nhận gradient từ các loss tương ứng. Ngược lại, EMA Teacher được cập nhật bằng trung bình mũ của Student, còn Prototype Distribution Bank được cập nhật bằng detached feature statistics. Cách tách này tránh việc statistical memory trở thành một bộ tham số bị tối ưu trực tiếp theo loss tại một minibatch.

Ký hiệu Student parameters là θ, Boundary Head là ψ, Trust Estimator là ϕ và EMA Teacher là $\bar{\theta}$. Sau optimizer step:

$$\theta \leftarrow \theta - \eta_\theta \nabla_\theta \mathcal{L}_{total}, \qquad \psi \leftarrow \psi - \eta_\psi \nabla_\psi \mathcal{L}_{total}, \qquad \phi \leftarrow \phi - \eta_\phi \nabla_\phi \mathcal{L}_{trust},$$

Teacher được cập nhật bằng

$$\bar{\theta} \leftarrow \alpha_T \bar{\theta} + (1 - \alpha_T)\theta.$$

Bank statistics (π, μ, Σ) nhận các feature đã `detach()` và cập nhật theo running-statistics/EMA. Temporal memory cũng là state không gradient. Do đó computational graph của CRISP-DA có ba loại state: trainable neural parameters, EMA network parameters, và non-parametric memory statistics.

4.4 Hai resolution cần phân biệt trong toàn pipeline

CRISP-DA làm việc đồng thời ở image resolution (H, W) và dense-feature resolution (H', W'), thường $H' = H/4$, $W' = W/4$. Segmentation logits có thể được upsample về (H, W) để tính source CE hoặc visualization, nhưng Prototype Bank, Trust Estimator và temporal memory nên hoạt động ở (H', W') để giảm memory/compute. Mọi evidence map trước khi concat phải được đưa về cùng hệ tọa độ và cùng spatial resolution:

$$Q, A, D, S, B \in \mathbb{R}^{B\times 1\times H'\times W'}.$$

Nếu augmentation có geometric transformation như crop/resize/flip, probability maps và boundary maps phải được inverse-warp hoặc forward-warp về cùng coordinate system trước khi tính consistency. Đây là yêu cầu triển khai bắt buộc; nếu bỏ qua alignment, a_i và s_i sẽ phản ánh sai khác hình học thay vì uncertainty thực sự.

5 Student Segmentation Network

5.1 Vai trò

Student là model chính được tối ưu trực tiếp bằng gradient và là model duy nhất bắt buộc giữ lại khi inference. Phiên bản canonical của tài liệu sử dụng MiT-B5/SegFormer-style encoder–decoder [1] để tách contribution của CRISP-DA khỏi foundation-model pretraining quá mạnh.

5.2 Input và output

Input:

$$x \in \mathbb{R}^{B\times 3\times H\times W}.$$

Output chính:

$$p \in [0,1]^{B\times C\times H\times W},$$

và dense feature dùng cho prototype/trust:

$$F^{dense} \in \mathbb{R}^{B\times D\times H/4\times W/4}, \qquad D = 256.$$

5.3 Chi tiết MiT-B5 encoder

Bảng 1: Cấu hình canonical của MiT-B5 trong CRISP-DA.

Stage	Resolution	Embed dim	Blocks	Heads	SR ratio	Patch/stride
1	$H/4 \times W/4$	64	3	1	8	$7/4$
2	$H/8 \times W/8$	128	6	2	4	$3/2$
3	$H/16 \times W/16$	320	40	5	2	$3/2$
4	$H/32 \times W/32$	512	3	8	1	$3/2$

Encoder tạo bốn feature maps:

$$F_1 \in \mathbb{R}^{B\times 64\times H/4\times W/4},$$

$$F_2 \in \mathbb{R}^{B\times 128\times H/8\times W/8},$$
$$F_3 \in \mathbb{R}^{B\times 320\times H/16\times W/16},$$
$$F_4 \in \mathbb{R}^{B\times 512\times H/32\times W/32}.$$

5.4 Dense projection và fusion

Mỗi F_l được chiếu về $D = 256$ bằng 1×1 convolution/linear projection. Các feature ở stage 2–4 được bilinear upsample lên độ phân giải $H/4 \times W/4$, rồi concatenate:

$$F^{cat} = \text{Concat}(P_1(F_1), \uparrow P_2(F_2), \uparrow P_3(F_3), \uparrow P_4(F_4)) \in \mathbb{R}^{B\times 1024\times H/4\times W/4}.$$

Sau đó dùng fusion layer:

$$F^{dense} = \text{GELU}(\text{Norm}(\text{Conv}_{1\times 1}^{1024\to 256}(F^{cat}))).$$

5.5 Segmentation head

Segmentation head canonical:

$$256 \xrightarrow{3\times 3} 256 \xrightarrow{\text{Norm+GELU}} 256 \xrightarrow{\text{Dropout}(0.1)} C.$$

Logits ở $H/4 \times W/4$ được bilinear upsample về $H \times W$ rồi softmax theo chiều class.

5.6 Source supervised learning

Với source image:

$$p_s = f_\theta(x_s),$$

loss cơ sở:

$$\mathcal{L}_{\text{sup}} = -\frac{1}{|\Omega|}\sum_{i\in\Omega} \log p_{s,i,y_{s,i}}.$$

6 EMA Teacher Network

6.1 Kiến trúc và cập nhật

EMA teacher có cùng kiến trúc encoder + segmentation head với student, theo nguyên lý Mean Teacher [5]. Teacher không được cập nhật bằng gradient. Sau mỗi optimizer step:

$$\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta,$$

trong đó giá trị khởi đầu hợp lý là $\alpha = 0.999$.

6.2 Input

Teacher nhận weakly augmented target views:

$$x_t^{w1} = a_{w1}(x_t), \qquad x_t^{w2} = a_{w2}(x_t).$$

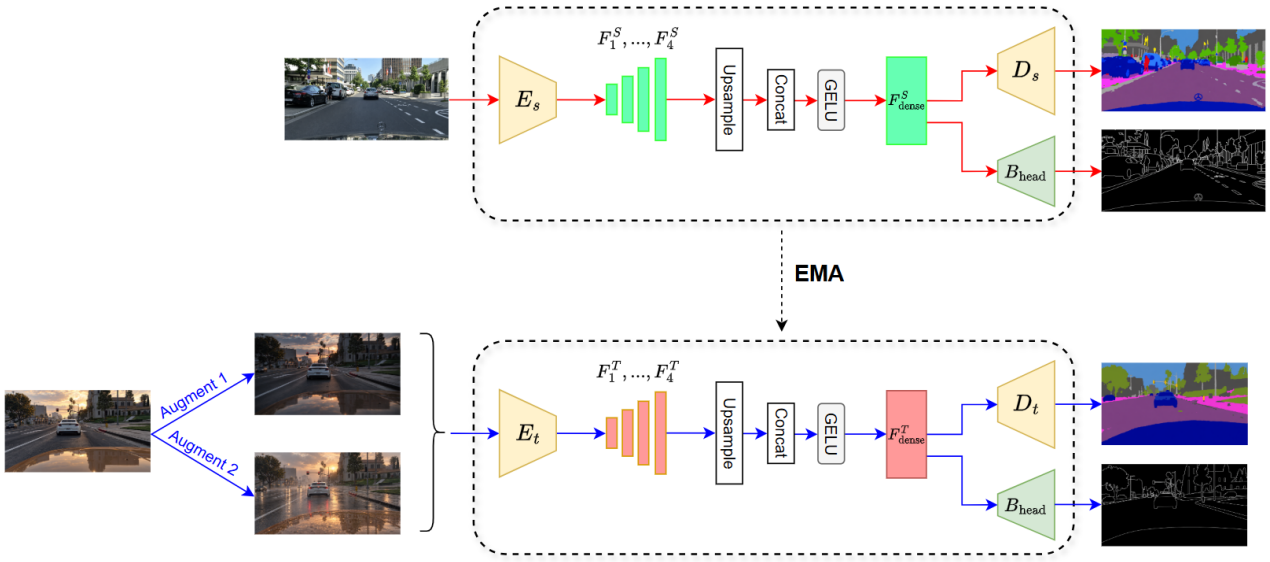
Weak augmentation nên giữ nguyên semantic structure: resize/crop hợp lệ, horizontal flip, color jitter nhẹ, brightness/contrast nhẹ. Nếu hai view có geometric transform khác nhau, prediction phải được warp về cùng hệ tọa độ trước khi so sánh consistency.

6.3 Output

Teacher sinh:

$$p_t^{T,1} = \text{softmax}(f_{\bar{\theta}}(x_t^{w1})), \tag{1}$$
$$p_t^{T,2} = \text{softmax}(f_{\bar{\theta}}(x_t^{w2})), \tag{2}$$
$$F_t^T = E_{\bar{\theta}}^{dense}(x_t^{w1}), \tag{3}$$
$$\tilde{y}_i = \arg\max_c p_{t,i,c}^{T,1}. \tag{4}$$

$\tilde{y}$ chỉ là raw pseudo-label; chưa được dùng trực tiếp để train student.



Hình 2: Minh họa Student/EMA-Teacher backbone và Boundary Head. Source branch tạo dense semantic feature, segmentation output và boundary output; target được tạo thành nhiều weak views và đưa qua EMA Teacher. Hình mang tính sơ đồ minh họa cho luồng dữ liệu, không quy định duy nhất một backbone cụ thể.

6.4 Teacher–student view construction và geometric alignment

Đối với mỗi target image x_t, CRISP-DA tách augmentation thành hai mức. Weak views $x_t^{w_1}, x_t^{w_2}$ giữ semantic content và geometry tương đối ổn định, ví dụ resize/crop đồng bộ, horizontal flip, mild color jitter hoặc brightness change. EMA Teacher chạy trên weak view để tạo pseudo-label và evidence ổn định. Strong view x_t^s có thể thêm stronger color jitter, blur, masking hoặc ClassMix/CutMix-style perturbation và được đưa vào Student.

Nếu hai weak views có cùng geometry, consistency được tính trực tiếp. Nếu có transform hình học T_1, T_2, prediction phải được đưa về canonical coordinates:

$$\bar{p}^{w_1} = T_1^{-1}(p^{w_1}), \qquad \bar{p}^{w_2} = T_2^{-1}(p^{w_2}).$$

Khi đó augmentation consistency mới được tính trên cùng pixel support. Tương tự, hard/soft targets sinh trên weak view phải được warp sang strong-view coordinates trước khi tính $\mathcal{L}_{\text{hard}}$ hoặc $\mathcal{L}_{\text{soft}}$. Việc này đặc biệt quan trọng với crop, flip và scale jitter; color-only augmentation không cần geometric warp.

6.5 Teacher outputs được dùng ở đâu?

EMA Teacher không chỉ tạo một segmentation map. Ở mỗi weak target view, nó cung cấp dense feature F_{dense}^T, logits z_t^T, probabilities p_t^T, raw pseudo-label $\tilde{Y}_t$, và nếu Boundary Head được mirror sang teacher thì có boundary probability B_t^T. Các output này tách thành nhiều nhánh: p_t^T tạo confidence q_i và augmentation consistency a_i; F_{dense}^T query Prototype Bank; p_t^T cập nhật temporal memory; B_t^T cung cấp spatial evidence; $\tilde{Y}_t$ xác định predicted class cho class-conditioned calibration và BPCC statistics.

7 Boundary Head

7.1 Motivation

Boundary và thin/small structures thường có confidence thấp hơn interior pixels. Nếu tất cả pixel dùng một reliability policy giống nhau, self-training dễ bỏ qua chính những vùng khó nhất. Boundary Head cung cấp spatial-role evidence cho Trust Estimator và thêm một objective riêng cho boundary.

7.2 Layer design

Boundary Head nhận $F^{dense} \in \mathbb{R}^{B\times 256\times H/4\times W/4}$:

$$\begin{aligned}&\text{Conv}_{3\times3}(256 \to 128) \to \text{Norm} \to \text{GELU} \to \\ &\text{Conv}_{3\times3}(128 \to 64) \to \text{Norm} \to \text{GELU} \to \\ &\text{Conv}_{1\times1}(64 \to 1) \to \text{Sigmoid} \to \text{Upsample}\ (H, W).\end{aligned}$$

Output:

$$b(x) \in [0,1]^{B\times H\times W}.$$

7.3 Source boundary ground truth

Từ y_s, sinh binary boundary band bằng morphological gradient hoặc contour extraction:

$$B_s^{GT} = \text{Dilate}(y_s) - \text{Erode}(y_s).$$

Boundary band có thể mở rộng 3–5 pixel tùy resolution.

Loss nguồn:

$$\mathcal{L}_{\text{bnd}}^{S} = \text{BCE}(b_s, B_s^{GT}) + \lambda_D \,\text{DiceLoss}(b_s, B_s^{GT}).$$

7.4 Target spatial role

EMA boundary head sinh b_t. Có thể đưa trực tiếp scalar b_i vào Trust Estimator hoặc định nghĩa hard role:

$$\rho_i = \begin{cases} \text{boundary}, & b_i > \tau_b, \\ \text{interior}, & \text{ngược lại.} \end{cases}$$

8 Class-Conditional Multi-Prototype Distribution Bank

8.1 Vai trò

Bank là memory module, không phải neural network. Nó mô hình hóa mỗi semantic class bằng nhiều mode thay vì một centroid duy nhất. Mục tiêu là xử lý intra-class multimodality: cùng class có thể thay đổi mạnh theo khoảng cách, ánh sáng, viewpoint, occlusion, thời tiết và domain.

8.2 Representation

Với class c, bank lưu K_c mode:

$$\mathcal{Q}_c = \{(\pi_{ck}, \mu_{ck}, \Sigma_{ck})\}_{k=1}^{K_c}.$$

Canonical config:

$$K_c = 3, \qquad \mu_{ck} \in \mathbb{R}^{256}, \qquad \Sigma_{ck} \in \mathbb{R}^{256}$$

với diagonal covariance để giảm chi phí.

8.3 Source bootstrap

Từ source dense features:

$$\mathcal{F}_c^S = \{f_i^S \mid y_{s,i} = c\},$$

thực hiện mini-batch hoặc offline K-means để khởi tạo μ_{ck}. Trong mỗi cluster, tính diagonal variance và mixture weight:

$$\pi_{ck} = \frac{N_{ck}}{\sum_j N_{cj}}.$$

8.4 Distribution compatibility

Nếu teacher dự đoán target pixel i là class c, tính Mahalanobis distance tới từng mode:

$$D_{ick} = (f_i^T - \mu_{ck})^\top \Sigma_{ck}^{-1} (f_i^T - \mu_{ck}).$$

Lấy mode gần nhất:

$$D_i^c = \min_k D_{ick}.$$

Chuyển thành compatibility:

$$d_i = \exp(-D_i^c / T_d) \in (0, 1].$$

8.5 Prototype posterior trên toàn bộ class

Để phục vụ REVISE, cần score cho mọi class:

$$S_i(c) = \log \sum_{k=1}^{K_c} \pi_{ck} \exp(-\tfrac{1}{2} D_{ick}),$$

$$p_i^{proto}(c) = \text{softmax}_c \, S_i(c),$$
$$c_i^{proto} = \arg\max_c p_i^{proto}(c).$$

8.6 Online update: source bootstrap và reliable-target adaptation

Distribution Bank không được xây lại từ đầu ở mỗi iteration; cách mô hình hóa multi-prototype/GMM có prior art gần với ProtoGMM [9], còn CRISP-DA tập trung vào reliability-gated bank adaptation. Sau khi các feature của batch hiện tại được gán vào class c và mode k, ta tính batch statistics

$$\hat{\mu}_{ck} = \frac{1}{N_{ck}} \sum_{i \in \mathcal{J}_{ck}} f_i, \qquad \hat{\Sigma}_{ck} = \frac{1}{N_{ck}} \sum_{i \in \mathcal{J}_{ck}} (f_i - \hat{\mu}_{ck})(f_i - \hat{\mu}_{ck})^\top,$$

$$\hat{\pi}_{ck} = \frac{N_{ck} + \alpha_0}{N_c + K_c \alpha_0},$$

trong đó $\mathcal{J}_{ck}$ là tập feature được gán vào mode k của class c, $N_{ck} = |\mathcal{J}_{ck}|$, $N_c = \sum_k N_{ck}$, và α_0 là hệ số smoothing nhỏ cho mixture weight. Khi dùng diagonal covariance, Σ_{ck} chỉ lưu phương sai theo từng feature dimension thay vì ma trận $D \times D$ đầy đủ.

Các statistics được cập nhật bằng exponential moving average (EMA), hoàn toàn ngoài gradient:

$$\mu_{ck}^{(t)} = m \mu_{ck}^{(t-1)} + (1-m) \hat{\mu}_{ck}^{(t)},$$

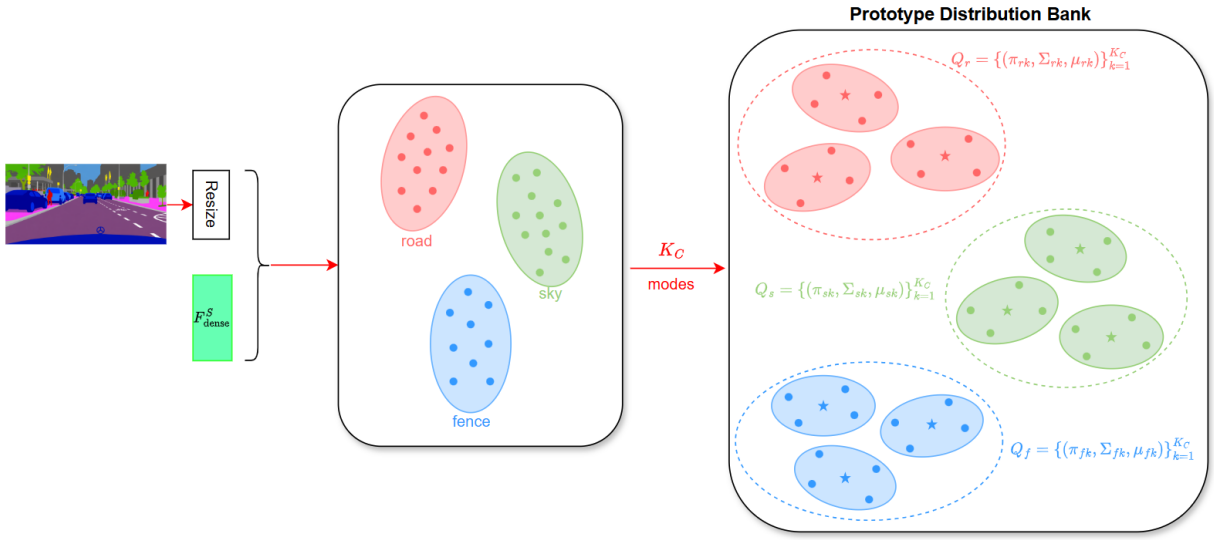
$$\Sigma_{ck}^{(t)} = m \Sigma_{ck}^{(t-1)} + (1-m) \hat{\Sigma}_{ck}^{(t)}, \qquad \pi_{ck}^{(t)} = m \pi_{ck}^{(t-1)} + (1-m) \hat{\pi}_{ck}^{(t)},$$

sau đó chuẩn hóa $\sum_k \pi_{ck} = 1$. Một giá trị khởi đầu hợp lý là $m \approx 0.99$.

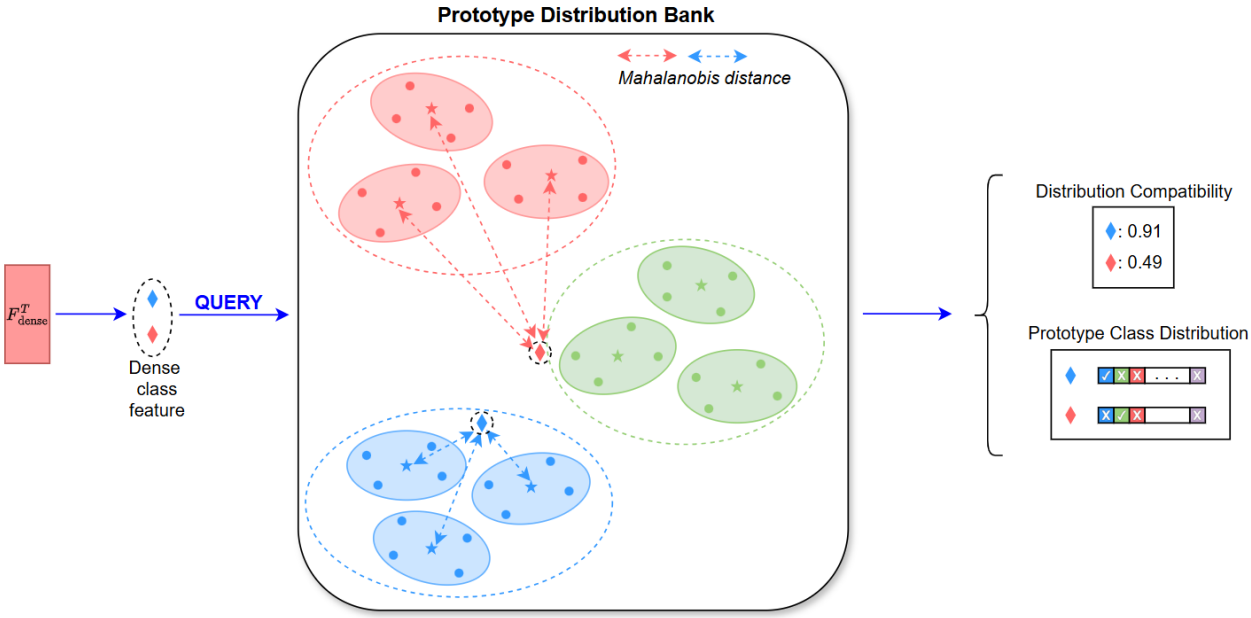
Điểm quan trọng là Bank **không source-only trong toàn bộ training**. Source features có ground truth được dùng để bootstrap và tiếp tục neo semantic distribution; target features được phép cập nhật Bank khi chúng đã vượt qua reliability filtering. Ở giai đoạn đầu chỉ source cập nhật Bank; sau warm-up, các target pixels ở trạng thái ACCEPT có thể tham gia; ở giai đoạn muộn có thể bổ sung một tập high-confidence REVISE với trọng số thấp hơn. SOFT và ABSTAIN không cập nhật Bank để tránh prototype contamination. Nếu gộp source và target trong cùng batch statistics, có thể dùng

$$\hat{\mu}_{ck} = \frac{\sum_{i \in S_{ck}} f_i^S + \lambda_T \sum_{j \in T_{ck}^{\text{rel}}} f_j^T}{N_{S,ck} + \lambda_T N_{T,ck}}, \qquad 0 < \lambda_T < 1,$$

để source GT giữ vai trò anchor mạnh hơn target pseudo-label. Mọi feature dùng cho Bank phải được `detach` khỏi computational graph.



Hình 3: Minh họa xây dựng Class-Conditional Multi-Prototype Distribution Bank từ dense feature: mỗi semantic class được mô hình hóa bởi nhiều mode với π_{ck}, μ_{ck} và Σ_{ck}.



Hình 4: Minh họa target-query: target dense feature được đối chiếu với các class-wise modes để tạo Distribution Compatibility d_i và Prototype Class Distribution p_i^{proto}.

8.7 Mode assignment trong online update

Sau bootstrap, mỗi incoming feature không chỉ cần biết class c mà còn phải được gán vào mode k. Với source feature, class assignment lấy trực tiếp từ ground truth; với target feature, class assignment chỉ dùng refined/reliable label đã qua Four-State Policy. Mode có thể được chọn bằng posterior lớn nhất hoặc khoảng cách Mahalanobis nhỏ nhất:

$$k_i^* = \arg\min_k D_{ick} \quad \text{hoặc} \quad k_i^* = \arg\max_k P(k \mid f_i, c).$$

Sau assignment, chỉ các mode có đủ sample trong minibatch mới cập nhật thống kê. Nếu N_{ck} quá nhỏ, update có thể bị skip để tránh variance estimate bất ổn. Với diagonal covariance, nên dùng variance floor ϵ_Σ:

$$\hat{\Sigma}_{ck} \leftarrow \max(\hat{\Sigma}_{ck}, \epsilon_\Sigma),$$

áp dụng theo từng feature dimension.

8.8 Reliable-target update và contamination control

CRISP-DA không phải source-only bank. Source chỉ tạo semantic anchor sạch ở giai đoạn đầu; mục tiêu sau warm-up là để bank thích nghi dần sang target. Tuy nhiên, tất cả target features không được update trực tiếp. Gọi $\mathcal{J}_{ck}^{S}$ là source features của class/mode (c, k) và $\mathcal{J}_{ck}^{T,A}$ là target features thuộc state ACCEPT. Batch mean có thể ghép hai domain bằng target weight $\lambda_T \in (0, 1]$:

$$\hat{\mu}_{ck}^{mix} = \frac{\sum_{i \in \mathcal{J}_{ck}^{S}} f_i^S + \lambda_T \sum_{j \in \mathcal{J}_{ck}^{T,A}} f_j^T}{|\mathcal{J}_{ck}^{S}| + \lambda_T |\mathcal{J}_{ck}^{T,A}| + \varepsilon}.$$

Covariance và occupancy được tính tương tự với cùng weighting. Ở middle stage, chỉ ACCEPT target được phép đóng góp. Ở late stage, một subset REVISE có thể được thêm nếu r_i, $p_i^{proto}(c_1)$ và Δ_i^{proto} đều vượt ngưỡng nghiêm ngặt; khi đó nên dùng trọng số nhỏ hơn ACCEPT. SOFT và ABSTAIN không update bank vì label assignment của chúng chưa đủ chắc chắn.

Điểm này tạo một feedback loop có kiểm soát. Bank hỗ trợ Trust Estimator và Four-State Policy; Policy lại quyết định feature nào đủ sạch để quay lại update Bank. Nếu gate quá lỏng, lỗi pseudo-label có thể tự củng cố; nếu gate quá chặt, bank không thích nghi được với target. Vì vậy tỷ lệ target samples tham gia bank, average compatibility và class-wise mode occupancy nên được log trong training.

8.9 Stabilization của mixture weights và rare modes

Mixture weight π_{ck} phản ánh occupancy tương đối của mode k trong class c. Để rare mode không biến mất do một vài minibatch không chứa sample, dùng Dirichlet smoothing

$$\hat{\pi}_{ck} = \frac{N_{ck} + \alpha_0}{N_c + K_c \alpha_0},$$

rồi EMA và renormalize

$$\pi_{ck} \leftarrow \frac{\pi_{ck}}{\sum_j \pi_{cj}}.$$

Có thể đặt lower floor π_{min} trước khi normalize. Nếu một mode liên tục có occupancy rất thấp, phiên bản V1 nên giữ số mode cố định thay vì dynamic split/merge để tránh tăng độ phức tạp. Dynamic mode birth/death có thể là extension sau khi baseline ổn định.

8.10 Outputs chính xác của Bank

Với mỗi target location i, Bank nên trả tối thiểu hai output. Thứ nhất là scalar compatibility d_i đối với class mà Teacher đang dự đoán; đây là input cho MTE. Thứ hai là full prototype class distribution $p_i^{proto} \in \mathbb{R}^C$; output này đi trực tiếp tới Four-State Policy để phục vụ REVISE và SOFT. Việc tách hai output tránh trộn vai trò: MTE cần biết "Teacher class có phù hợp với distribution không?", còn Four-State Policy cần biết "nếu không phải Teacher class, class alternative nào được Bank hỗ trợ?".

9 Multi-Evidence Trust Estimator

9.1 Mục tiêu và input/output

Multi-Evidence Trust Estimator (MTE) ước lượng mức độ đáng tin của raw target pseudo-label tại từng spatial location. Thay vì đồng nhất raw confidence với correctness, MTE học một ánh xạ

$$r_i = G_\phi(q_i, a_i, d_i, s_i, b_i, e_{c_i}) \in [0, 1],$$

trong đó r_i được diễn giải như một calibrated reliability score, xấp xỉ xác suất pseudo-label hiện tại là đúng dưới các evidence quan sát được. Nếu $F_{dense}^T \in \mathbb{R}^{B \times D \times H' \times W'}$, mỗi scalar evidence được biểu diễn thành một map $B \times 1 \times H' \times W'$. Sau khi concatenate năm evidence và class embedding $d_c = 16$, input có shape $B \times (5 + d_c) \times H' \times W' = B \times 21 \times H' \times W'$; output là reliability map $R_t \in [0, 1]^{B \times 1 \times H' \times W'}$.

9.2 Teacher confidence q_i: predictive certainty

Với teacher probability $p_i^T \in [0, 1]^C$,

$$q_i = \max_c p_{i,c}^T.$$

Confidence được giữ lại vì nó phản ánh mức certainty tức thời của Teacher, nhưng không được dùng đơn độc: dưới domain shift, model có thể confidently wrong. Vì vậy q_i trả lời "Teacher tự tin đến đâu?", chưa đủ trả lời "Teacher có đúng không?".

9.3 Augmentation consistency a_i: perturbation stability

Hai weak views $x_t^{w_1}$ và $x_t^{w_2}$ tạo hai distribution p_i^{T,w_1} và p_i^{T,w_2}. Sau khi warp về cùng coordinate system,

$$a_i = 1 - \mathrm{JS}(p_i^{T,w_1}, \hat{p}_i^{T,w_2}),$$

được normalize về $[0, 1]$. Pseudo-label đáng tin nên ít thay đổi dưới perturbation nhẹ; a_i vì vậy cung cấp evidence mà raw confidence không thể hiện.

9.4 Distribution compatibility d_i: semantic-feature agreement

d_i được lấy trực tiếp từ Class-Conditional Multi-Prototype Distribution Bank. Nếu Teacher dự đoán class c nhưng target feature nằm xa mọi mode của c, d_i thấp và cho biết prediction có thể "confident but distributionally inconsistent". Đây là evidence giúp phát hiện lỗi do domain shift trong feature space.

9.5 Temporal stability s_i: consistency qua thời gian

Với target image ID j, ta duy trì low-resolution EMA probability memory

$$M_j^{(t)} = \gamma M_j^{(t-1)} + (1-\gamma) p_j^{T,(t)},$$

rồi tính

$$s_i = 1 - \mathrm{JS}(p_i^{T,(t)}, M_i^{(t-1)}).$$

Prediction lặp lại ổn định qua nhiều checkpoint/iteration được xem đáng tin hơn prediction dao động giữa nhiều class. Canonical $\gamma \approx 0.9$; memory nên lưu ở $H/8$ hoặc $H/16$ và FP16 để giảm RAM.

9.6 Boundary evidence b_i: spatial difficulty

Boundary Head cung cấp $b_i \in [0, 1]$, là xác suất pixel nằm gần semantic boundary. Cùng một confidence có thể có reliability khác nhau ở object interior và boundary; do đó b_i giúp MTE học spatially conditioned calibration. Trong phiên bản cơ sở, dùng continuous b_i là đủ và không bắt buộc thêm một discrete role embedding riêng.

9.7 Class conditioning

Predicted class $c_i = \tilde{y}_i^T$ được ánh xạ qua embedding $e_{c_i} \in \mathbb{R}^{16}$. Điều này cho phép cùng một confidence/reliability pattern được diễn giải khác nhau giữa easy/head class và hard/tail class. Input cuối tại pixel i là

$$u_i = [q_i, a_i, d_i, s_i, b_i; e_{c_i}] \in \mathbb{R}^{21}.$$

9.8 Kiến trúc estimator

MTE có thể triển khai như cùng một MLP áp cho mọi spatial location, tương đương chuỗi 1×1 convolution:

$$21 \to 64 \to 32 \to 1,$$

với GELU, Dropout 0.1 và sigmoid cuối. Nhờ đó, $R_t = G_\phi(U_t)$ giữ nguyên spatial resolution $H' \times W'$ và có chi phí nhỏ so với backbone.

9.9 Huấn luyện MTE bằng source correctness calibration

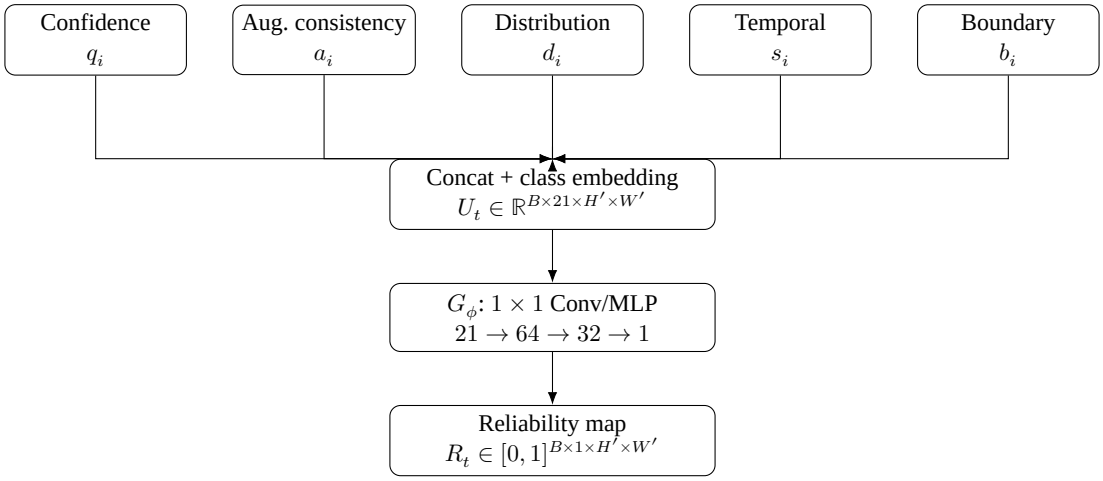
Target không có ground truth, vì vậy MTE không thể học trực tiếp từ target correctness. CRISP-DA tạo corrupted source $\hat{x}_s = T(x_s)$ bằng blur/fog/noise/brightness/color shift và tính cùng evidence vector trên các source samples bị dịch chuyển. Do source có label, correctness target được xác định chính xác:

$$g_i = \mathbf{1}[\hat{y}_{s,i} = y_{s,i}].$$

Trust loss là class-balanced BCE:

$$\mathcal{L}_{trust} = -\frac{1}{N} \sum_i \left[w_1 g_i \log r_i + w_0 (1 - g_i) \log(1 - r_i) \right].$$

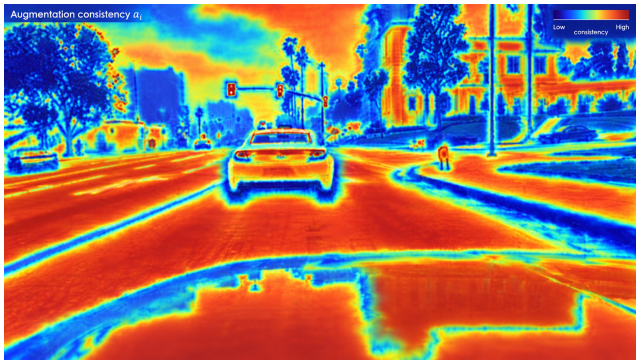
Cơ chế này không khẳng định corrupted source hoàn toàn mô phỏng target shift; đó là một giả thuyết calibration cần ablation. Tuy nhiên nó cung cấp supervised correctness signal mà không sử dụng target labels.



Hình 5: Multi-Evidence Trust Estimator: năm evidence bổ sung cho nhau được fuse thành reliability score thay vì dùng max-softmax confidence như một quyết định nhị phân.



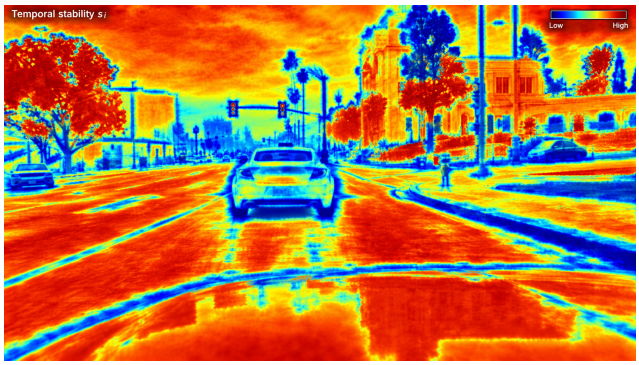
(a) Teacher confidence q_i.



(b) Augmentation consistency a_i.



(c) Distribution compatibility d_i.



(d) Temporal stability s_i.



(e) Boundary evidence b_i.

Hình 6: Minh họa trực quan các evidence maps của MTE. Đây là hình minh họa khái niệm cho cách biểu diễn signal theo không gian, không phải kết quả định lượng của một checkpoint CRISP-DA đã được huấn luyện.

9.10 Lý do chọn năm evidence và tính bổ sung của chúng

Năm evidence không được chọn chỉ để tăng số feature đầu vào. Mỗi signal nhằm bắt một failure mode khác nhau. q_i đo certainty nội tại của classifier nhưng dễ over-confident dưới domain shift. a_i đo local robustness đối với input perturbation, vì prediction đúng thường bền hơn trước weak augmentation. d_i là một kiểm tra ở feature/distribution space, có khả năng phản bác một prediction dù softmax confidence cao. s_i đưa thông tin xuyên thời gian, phân biệt một prediction ổn định nhiều iteration với một prediction vừa thay đổi. b_i mô tả spatial difficulty: cùng confidence nhưng boundary pixel thường có xác suất sai cao hơn interior pixel. Class embedding thêm context vì calibration curve của các semantic class không giống nhau.

Ví dụ một pixel có thể có $(q_i, a_i, d_i, s_i, b_i) = (0.95, 0.91, 0.28, 0.87, 0.15)$. Confidence, augmentation và temporal signals đều cao nhưng distribution compatibility rất thấp; đây là mẫu điển hình của "confidently wrong" mà confidence-only filtering khó phát hiện. Ngược lại, một boundary pixel có $(0.73, 0.86, 0.82, 0.90, 0.94)$: confidence chỉ trung bình nhưng các evidence còn lại nhất quán, vì vậy estimator có thể gán reliability cao hơn một global threshold heuristic.

9.11 Pixel-wise architecture và lý do dùng pointwise Conv

Vì mỗi evidence đã là một scalar map được spatially aligned, nhiệm vụ MTE V1 chủ yếu là học nonlinear fusion tại từng location. Một chuỗi 1×1 convolution tương đương cùng một MLP được áp cho mọi pixel, giữ nguyên (H', W') và không trộn evidence giữa các spatial neighbors. Điều này làm diễn giải rõ: output r_i chỉ phụ thuộc vào evidence của chính location i và class embedding của nó. Nếu sau này muốn khai thác neighborhood context, có thể thay một layer bằng 3×3 depthwise/separable conv, nhưng điều đó nên được ablate riêng vì sẽ thay đổi giả thuyết từ reliability fusion sang spatial reasoning.

9.12 Calibration target, class imbalance và validation của Trust Estimator

Correctness label $g_i = \mathbf{1}[\hat{y}_{s,i} = y_{s,i}]$ thường rất mất cân bằng vì phần lớn source pixels có thể đúng sau pretraining. Do đó $\mathcal{L}_{\text{trust}}$ nên dùng class-balanced BCE, focal variant hoặc balanced sampling của correct/incorrect pixels. Quan trọng hơn, MTE cần được đánh giá như một reliability estimator chứ không chỉ như một auxiliary classifier. Các metric phù hợp gồm AUROC/AUPRC cho error detection, Expected Calibration Error (ECE), Brier score và reliability diagrams của r_i. Nếu r_i không được calibration tốt, downstream thresholds $\tau_{high,c}, \tau_{low,c}$ sẽ khó có ý nghĩa ổn định giữa class và epoch.

Ở target domain không có correctness label, nên MTE chỉ được dùng inference-style để sinh r_i; không backpropagate một target correctness loss không giám sát vào G_ϕ trong phiên bản cơ sở. Có thể fine-tune gián tiếp bằng consistency hoặc entropy objectives trong extension, nhưng cần tránh circularity: nếu Trust Estimator học từ chính pseudo-label mà nó đang đánh giá, confirmation bias có thể quay lại dưới dạng mới.

10 Four-State Pseudo-Label Policy

10.1 Vai trò, input và output

Four-State Policy là decision layer nằm giữa reliability estimation và Student supervision. Nó nhận raw Teacher distribution p_i^T, raw label $\tilde{y}_i^T$, calibrated reliability r_i, Prototype Class Distribution p_i^{proto}, prototype top class c_i^{proto}, cùng class-wise thresholds $\tau_{high,c}(t)$ và $\tau_{low,c}(t)$ từ BPCC. Output không phải một label map duy nhất mà gồm: hard target map Y_t^* cho ACCEPT/REVISE, soft target tensor P_t^{soft} cho SOFT, state masks M_A, M_R, M_S, M_X và optional pixel weights.

10.2 ACCEPT: giữ Teacher hard label

Nếu $r_i \geq \tau_{high,c}(t)$ và prototype evidence không phản đối Teacher mạnh, giữ

$$y_i^* = \tilde{y}_i^T.$$

ACCEPT là nhánh hard-supervision có độ tin cậy cao nhất và thường dùng weight 1.0.

10.3 REVISE: sửa hard pseudo-label bằng alternative evidence

Đặt

$$c_1 = \arg\max_c p_i^{proto}(c), \qquad c_2 = \arg\max_{c \neq c_1} p_i^{proto}(c).$$

Prototype margin là

$$\Delta_i^{proto} = p_i^{proto}(c_1) - p_i^{proto}(c_2).$$

Nếu $c_1 \neq \tilde{y}_i^T$, $p_i^{proto}(c_1) > \eta$, $\Delta_i^{proto} > \delta$, và Teacher reliability không nằm trong vùng cực cao, policy có thể sửa

$$y_i^* = c_1.$$

Margin ngăn việc sửa label khi Prototype Bank còn phân vân giữa hai class gần nhau. REVISE nên dùng weight $w_R < 1$ vì rủi ro cao hơn ACCEPT.

10.4 SOFT SUPERVISE: giữ uncertainty thay vì ép integer class

Nếu

$$\tau_{low,c}(t) < r_i < \tau_{high,c}(t)$$

và chưa đủ điều kiện REVISE, policy tạo một soft probability target thay vì integer class:

$$p_i^{soft} = \alpha_i p_i^T + (1-\alpha_i) p_i^{proto}, \qquad p_i^{soft} \in [0,1]^C,$$

$$\sum_c p_{i,c}^{soft} = 1.$$

Có thể chọn $\alpha_i = r_i$ hoặc một mapping calibrated. Khi training nhánh SOFT, Student học trực tiếp distribution này bằng KL/soft cross-entropy; không cần ép p_i^{soft} thành arg max vì làm như vậy sẽ đánh mất uncertainty.

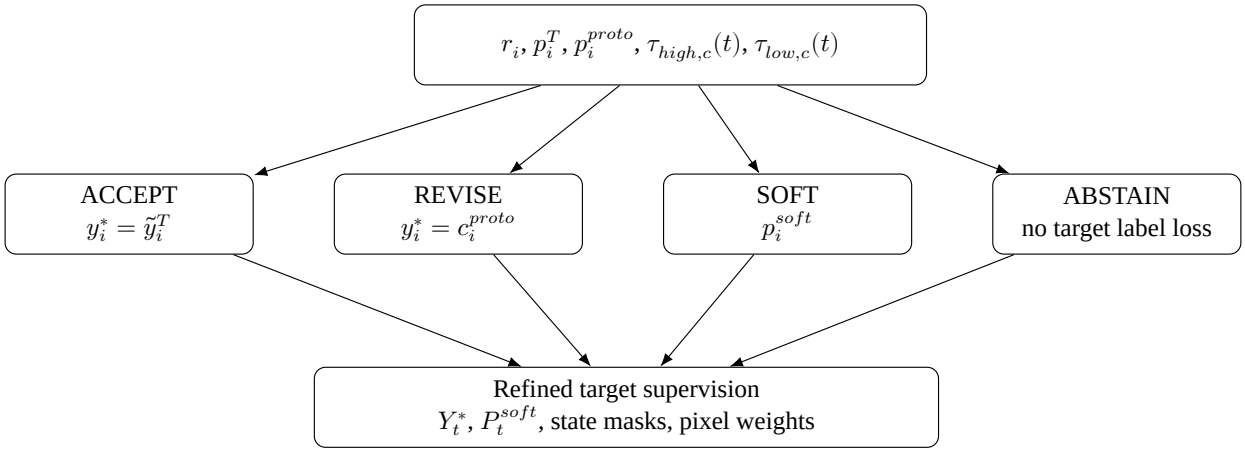
10.5 ABSTAIN: không dùng pseudo-supervision tại iteration hiện tại

Nếu $r_i \le \tau_{low,c}(t)$ hoặc các evidence xung đột mạnh, đặt $M_{X,i} = 1$ và $M_{A,i} = M_{R,i} = M_{S,i} = 0$. Pixel không bị loại vĩnh viễn: khi Teacher, Bank hoặc temporal evidence cải thiện, nó có thể chuyển từ ABSTAIN sang SOFT/ACCEPT ở iteration sau.

Một policy logic tối giản có thể viết

$$\text{State}_i = \begin{cases} \text{REVISE}, & c_1 \neq \tilde{y}_i^T,\ p_i^{proto}(c_1) > \eta,\ \Delta_i^{proto} > \delta,\ r_i < \tau_{very-high,c}, \\ \text{ACCEPT}, & r_i \ge \tau_{high,c}(t) \text{ và không có prototype conflict mạnh}, \\ \text{SOFT}, & \tau_{low,c}(t) < r_i < \tau_{high,c}(t), \\ \text{ABSTAIN}, & r_i \le \tau_{low,c}(t) \text{ hoặc evidence conflict mạnh}. \end{cases}$$

REVISE được kiểm tra trước SOFT để một pixel reliability trung bình nhưng có alternative prototype evidence rất rõ không bị mặc định rơi vào soft branch.



Hình 7: Four-State Pseudo-Label Policy. ACCEPT và REVISE tạo hard targets; SOFT giữ distribution uncertainty; ABSTAIN tạm thời không tạo pseudo-label supervision.

10.6 Thứ tự ra quyết định và các điều kiện xung đột

Policy cần có thứ tự kiểm tra xác định để tránh một pixel đồng thời thỏa nhiều nhánh. Một lựa chọn hợp lý là: (1) kiểm tra REVISE chỉ khi có prototype alternative đủ mạnh và Teacher không ở vùng "very-high trust"; (2) nếu không REVISE, kiểm tra ACCEPT; (3) sau đó SOFT; (4) còn lại ABSTAIN. Lý do kiểm tra REVISE trước SOFT là một pixel reliability trung bình có thể mang evidence sửa label rất rõ; nếu áp threshold trước, nó sẽ bị đưa vào SOFT dù alternative class gần như chắc chắn. Ngược lại, nếu r_i cực cao và prototype chỉ bất đồng nhẹ, không nên sửa Teacher vì bank cũng có thể đang source-biased.

Có thể thêm conflict score

$$\chi_i = \mathbf{1}[c_i^{proto} \neq \tilde{y}_i^T] \cdot p_i^{proto}(c_i^{proto})$$

hoặc divergence $JS(p_i^T, p_i^{proto})$ để phát hiện disagreement mạnh. Phiên bản đầu nên giữ rule đơn giản để ablation dễ giải thích; các conflict score phức tạp chỉ nên thêm khi có bằng chứng thực nghiệm.

10.7 Supervision masks và weighting

Sau routing, policy tạo các mask nhị phân $M_A, M_R, M_S, M_X \in \{0,1\}^{B\times H'\times W'}$ thỏa

$$M_A + M_R + M_S + M_X = 1.$$

Hard mask là $M_{hard} = M_A + M_R$, soft mask là $M_{soft} = M_S$. Có thể dùng pixel weight

$$w_i = \begin{cases} 1, & i \in A, \\ w_R, & i \in R, \\ w_S(r_i), & i \in S, \\ 0, & i \in X, \end{cases}$$

trong đó $0 < w_R < 1$ phản ánh rủi ro cao hơn của label correction và $w_S(r_i)$ có thể tăng tuyến tính theo reliability. Các loss phải normalize theo tổng weight thực tế, không chia cố định cho toàn bộ HW, để batch có nhiều ABSTAIN không làm giảm scale gradient ngoài ý muốn.

10.8 Liên hệ giữa policy và Bank update

Four-State Policy không chỉ kiểm soát Student supervision mà còn đóng vai trò gate cho non-parametric memory. ACCEPT pixels có thể update bank sau warm-up; REVISE chỉ update ở late stage khi alternative evidence đủ mạnh; SOFT và ABSTAIN không update. Nhờ đó cùng một reliability decision được dùng nhất quán ở hai nơi: quyết định cách Student học và quyết định feature nào được phép thay đổi semantic reference của Bank. Đây là điểm quan trọng của CRISP-DA vì nếu Student supervision được lọc kỹ nhưng Bank vẫn nhận mọi target feature, contamination có thể xảy ra qua một đường vòng.

11 Balanced Progressive Coverage Curriculum

11.1 Mục tiêu và input/output

Balanced Progressive Coverage Curriculum (BPCC) là một non-parametric training controller, không phải neural network. Nó không sinh pseudo-label; nó quyết định ở từng giai đoạn training, mỗi semantic class nên được phép sử dụng bao nhiêu pseudo-label. Input gồm reliability map $R_t = \{r_i\}$, raw Teacher class map $\tilde{Y}_t$, training progress t/T, và running statistics theo class. Output chính là hai threshold động

$$\tau_{high,c}(t), \qquad \tau_{low,c}(t),$$

được gửi tới Four-State Policy.

11.2 Class-wise coverage và reliability

Với class c, định nghĩa

$$\kappa_c^{(t)} = \frac{N_{used,c}^{(t)}}{N_{pred,c}^{(t)} + \varepsilon},$$

trong đó $N_{pred,c}$ là số target pixels Teacher dự đoán thuộc class c, còn $N_{used,c}$ là số pixels class đó hiện được dùng làm hard supervision, thường ACCEPT + REVISE. Ta cũng duy trì reliability trung bình bằng EMA

$$\bar{r}_c^{(t)} = \beta_r \bar{r}_c^{(t-1)} + (1-\beta_r)\,\text{Mean}_{i:\tilde{y}_i=c}(r_i).$$

11.3 Progressive target coverage

Coverage mong muốn tăng dần theo training:

$$\kappa_c^*(t) = \kappa_{min,c} + (\kappa_{max,c} - \kappa_{min,c})g(t), \qquad g(t) \in [0,1].$$

Ở đầu training, κ^* thấp để ưu tiên precision; khi Teacher/Bank/Trust ổn định, κ^* tăng dần. Đây là nguyên tắc **high precision first → progressively expanded coverage**.

11.4 Class-balanced threshold controller

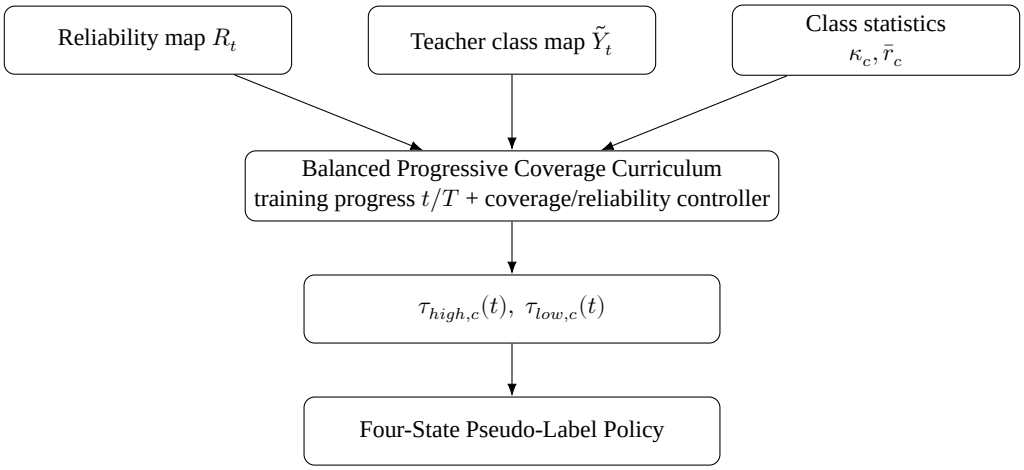
Một controller đơn giản là

$$\tau_{high,c}^{(t+1)} = \text{Clip}\left[\tau_{high,c}^{(t)} + \eta(\kappa_c^{(t)} - \kappa_c^*(t)) + \lambda \max(0, r_{min} - \bar{r}_c^{(t)})\right].$$

Nếu coverage của class thấp hơn target nhưng reliability vẫn tốt, thành phần $\kappa_c - \kappa_c^*$ âm làm threshold giảm và nhận thêm pseudo-label. Nếu coverage đã quá cao, threshold tăng. Nếu reliability trung bình thấp hơn r_{min}, safety term làm threshold tăng, ngăn controller hạ chuẩn chỉ để đạt quota. Để giảm số hyperparameter, có thể đặt

$$\tau_{low,c}(t) = \tau_{high,c}(t) - \Delta_\tau.$$

Nhờ class-specific thresholds, easy classes không chiếm toàn bộ target supervision và hard/tail classes có cơ hội mở rộng coverage khi evidence đủ tốt.



Hình 8: BPCC điều khiển threshold theo class và training stage. Module sử dụng statistics, không có trainable parameters.

11.5 Quantile controller: lựa chọn thay thế dễ triển khai

Ngoài controller dạng gradient-free update của $\tau_{high,c}$, một implementation V1 rất trực quan là class-wise quantile. Với class c, gom reliability scores trong một running window

$$\mathcal{R}_c = \{r_i : \tilde{y}_i = c\}.$$

Nếu desired hard coverage ở epoch t là $\kappa_c^*(t)$, đặt

$$\tau_{high,c}(t) = Q_{1-\kappa_c^*(t)}(\mathcal{R}_c),$$

trong đó Q_p là p-quantile. Điều này tương đương chọn top $\kappa_c^*(t)$ reliability pixels của từng class. Để không "ép quota" khi toàn bộ class đang kém, dùng safety floor

$$\tau_{high,c}(t) \leftarrow \max\{\tau_{high,c}(t), \tau_{safe}\}.$$

Cách quantile có ưu điểm ít hyperparameter, dễ kiểm tra và trực tiếp đảm bảo class-wise coverage; nhược điểm là threshold có thể dao động nếu class xuất hiện ít, do đó cần running window hoặc EMA histogram.

11.6 Feedback loop giữa BPCC và Four-State Policy

BPCC và Four-State Policy tạo một closed-loop controller. Policy ở iteration hiện tại tạo state counts $N_{A,c}, N_{R,c}, N_{S,c}, N_{X,c}$; BPCC dùng các counts này để cập nhật coverage κ_c và thresholds cho iteration/epoch tiếp theo. Khi model tốt hơn, reliability distribution dịch lên, nhiều pixel tự nhiên vượt $\tau_{high,c}$. Nếu một class mở rộng quá nhanh, threshold tăng; nếu class coverage thấp nhưng reliability tốt, threshold giảm nhẹ. Vì vậy progression điển hình của một pixel có thể là

$$\text{ABSTAIN} \rightarrow \text{SOFT} \rightarrow \text{ACCEPT},$$

nhưng không bị ép phải đi theo thứ tự này; một pixel có prototype alternative mạnh có thể chuyển sang REVISE.

11.7 Statistics nên log để kiểm tra curriculum

Để biết BPCC hoạt động đúng, mỗi epoch nên log theo class: κ_c, $\bar{r}_c$, $\tau_{high,c}$, $\tau_{low,c}$, tỷ lệ bốn states và pseudo-label precision trên một validation proxy nếu có. Nếu head classes có coverage tăng nhanh còn tail classes gần zero trong nhiều epoch, controller chưa đạt mục tiêu cân bằng. Ngược lại, nếu tail-class coverage tăng nhưng average reliability giảm mạnh, threshold schedule đang quá aggressive.

12 Target strong-view training

12.1 Strong augmentation

Từ ảnh gốc:

$$x_t^s = a_s(x_t),$$

trong đó a_s có thể gồm strong color jitter, blur, CutOut/masking và optional ClassMix. Tất cả geometric transforms phải được áp dụng đồng bộ lên $Y_t^*, P_t^{soft}, M_{hard}, M_{soft}$.

12.2 Student forward

$$p_t^S = f_\theta(x_t^s), \qquad F_t^S = E_\theta^{dense}(x_t^s), \qquad b_t^S = B_\psi(F_t^S).$$

13 Thiết kế các hàm loss

13.1 Source segmentation loss

$$\mathcal{L}_{\text{sup}} = CE(p_s, y_s).$$

Có thể thêm class weighting nếu source imbalance nghiêm trọng, nhưng nên giữ baseline đơn giản ở phiên bản đầu.

13.2 Hard target loss

Với ACCEPT và REVISE:

$$\mathcal{L}_{\text{hard}} = -\frac{1}{\sum_i M_{hard,i} + \varepsilon} \sum_i M_{hard,i} w_i \log p^S_{t,i,y_i^*}.$$

Đặt $w_i = 1$ cho ACCEPT và $w_i = w_R < 1$ cho REVISE.

13.3 Soft target loss

$$\mathcal{L}_{\text{soft}} = \frac{1}{\sum_i M_{soft,i} + \varepsilon} \sum_i M_{soft,i}\, \text{KL}(p_i^{soft} \| p^S_{t,i}).$$

13.4 Consistency loss

Warp teacher weak prediction sang strong geometry:

$$\hat{p}_i^T = \mathcal{T}_{w\to s}(p_i^{T,1}).$$

Reliability-weighted consistency:

$$\mathcal{L}_{\text{cons}} = \frac{1}{|\Omega|} \sum_i r_i\, \text{KL}(\hat{p}_i^T \| p_i^S).$$

Có thể mask ABSTAIN pixels nếu muốn conservative training.

13.5 Target boundary loss

Tạo semantic edge từ refined hard labels:

$$\hat{B}_t^{sem} = \partial Y_t^*.$$

Chỉ chọn boundary pixel khi semantic edge và EMA boundary head đồng thuận:

$$M_i^{bnd} = \mathbf{1}[\hat{B}_{t,i}^{sem} = 1 \wedge b_{t,i}^T > \tau_b].$$

Loss:

$$\mathcal{L}_{\text{bnd}}^T = \text{BCE}(b_t^S, \hat{B}_t^{sem}; M^{bnd}) + \lambda_D\, \text{DiceLoss}(b_t^S, \hat{B}_t^{sem}; M^{bnd}).$$

13.6 Prototype loss

Trên reliable target pixels:

$$\mathcal{L}_{\text{proto}} = \frac{1}{|\Omega_R|} \sum_{i \in \Omega_R} \min_k D_{i,y_i^*,k}.$$

Có thể thay bằng mixture NLL nếu muốn probabilistic formulation chặt hơn.

13.7 Trust calibration loss

$$\mathcal{L}_{\text{trust}} = \text{WBCE}(r_i^S, g_i),$$

với WBCE là weighted BCE trên corrupted source samples.

13.8 Total objective

Objective đầy đủ:

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda_{sb}\mathcal{L}_{\text{bnd}}^S + \lambda_h\mathcal{L}_{\text{hard}} + \lambda_s\mathcal{L}_{\text{soft}} + \lambda_{tb}\mathcal{L}_{\text{bnd}}^T + \lambda_c\mathcal{L}_{\text{cons}} + \lambda_p\mathcal{L}_{\text{proto}} + \lambda_r\mathcal{L}_{\text{trust}}}$$

Bảng 2: Ý nghĩa của các loss trong CRISP-DA.

Loss	Vai trò
$\mathcal{L}_{\text{sup}}$	Giữ semantic supervision sạch từ source.
$\mathcal{L}_{\text{bnd}}^S$	Học boundary prior từ source GT.
$\mathcal{L}_{\text{hard}}$	Học hard target labels từ ACCEPT/REVISE.
$\mathcal{L}_{\text{soft}}$	Học uncertain-but-informative target pixels mà không ép hard class.
$\mathcal{L}_{\text{bnd}}^T$	Cải thiện target boundary khi có đủ evidence đồng thuận.
$\mathcal{L}_{\text{cons}}$	Ép weak-teacher và strong-student predictions nhất quán.
$\mathcal{L}_{\text{proto}}$	Giữ target feature phù hợp với distribution của semantic class.
$\mathcal{L}_{\text{trust}}$	Học calibration mapping từ multi-evidence sang correctness probability.

13.9 Masking, normalization và gradient routing của từng loss

Các loss target chỉ được tính trên state tương ứng. Đặt $\Omega_A, \Omega_R, \Omega_S$ là tập pixel ACCEPT, REVISE và SOFT sau khi geometric warp sang strong-view coordinates. Hard loss nên normalize theo tổng hard weight:

$$\mathcal{L}_{\text{hard}} = -\frac{1}{\sum_{i \in \Omega_A \cup \Omega_R} w_i + \varepsilon} \sum_{i \in \Omega_A \cup \Omega_R} w_i \log p_{i,y_i^*}^S.$$

Soft loss tương tự:

$$\mathcal{L}_{\text{soft}} = \frac{1}{\sum_{i \in \Omega_S} w_i + \varepsilon} \sum_{i \in \Omega_S} w_i \, \text{KL}\left(p_i^{soft} \| p_i^S\right).$$

ABSTAIN pixels không đóng góp vào hai loss này. Nếu một minibatch không có pixel của một state, loss tương ứng đặt bằng 0 thay vì chia cho zero.

Teacher probabilities, prototype posterior và refined targets dùng làm target nên được `detach()` trước khi tính Student loss. Nếu không, gradient có thể chạy ngược vào module tạo pseudo-label và làm policy trở thành một đường tối ưu hóa không chủ ý. $\mathcal{L}_{\text{proto}}$ là ngoại lệ nếu được thiết kế để cập nhật Student feature encoder: prototype parameters vẫn detach, còn f_i^S / f_i^T giữ gradient. $\mathcal{L}_{\text{trust}}$ cập nhật G_ϕ trên source-calibration branch; target reliability scores dùng cho routing không cần gradient qua policy.

13.10 Weighted total objective và lịch bật loss

Không nhất thiết bật toàn bộ loss từ epoch đầu. Một lịch ổn định hơn là source bootstrap với $\mathcal{L}_{\text{sup}} + \lambda_{sb}\mathcal{L}_{\text{bnd}}^S + \lambda_r\mathcal{L}_{\text{trust}}$, sau đó bật $\mathcal{L}_{\text{hard}}$ ở conservative adaptation, rồi mới thêm $\mathcal{L}_{\text{soft}}$, $\mathcal{L}_{\text{cons}}$, $\mathcal{L}_{\text{proto}}$ và target boundary loss. Tổng loss ở full stage:

$$\mathcal{L}_{total} = \mathcal{L}_{\text{sup}} + \lambda_{sb}\mathcal{L}_{\text{bnd}}^S + \lambda_h\mathcal{L}_{\text{hard}} + \lambda_s\mathcal{L}_{\text{soft}} + \lambda_{tb}\mathcal{L}_{\text{bnd}}^T + \lambda_c\mathcal{L}_{\text{cons}} + \lambda_p\mathcal{L}_{\text{proto}} + \lambda_r\mathcal{L}_{\text{trust}}.$$

Các λ là trade-off coefficients; tài liệu này không xem một bộ số cụ thể là mặc định đã được xác nhận. Cần tune bằng source validation/unsupervised diagnostics và ablation trên benchmark chuẩn.

14 Quy trình training đầy đủ

14.1 Stage 0: preprocessing

Chuẩn hóa label mapping source/target, chọn crop size, sinh boundary GT cho source, định nghĩa weak/strong augmentation, xây cache/index cho temporal memory.

14.2 Stage 1: source bootstrap

Khoảng 15–25% tổng training budget:

1. Train Student Encoder + Segmentation Head bằng $\mathcal{L}_{\text{sup}}$.
2. Train Boundary Head bằng $\mathcal{L}_{\text{bnd}}^{S}$.
3. Thu source dense features và bootstrap Multi-Prototype Bank.
4. Train Trust Estimator bằng corrupted source và $\mathcal{L}_{\text{trust}}$.
5. Khởi tạo EMA teacher từ student.

14.3 Stage 2: conservative adaptation

Khoảng 15–25% tiếp theo. Chỉ bật ACCEPT và ABSTAIN hoặc bật REVISE với điều kiện rất chặt. Threshold cao, Bank update vẫn được neo bởi source và chỉ cho ACCEPT target tham gia. Mục tiêu là bảo vệ precision và tránh poisoning.

14.4 Stage 3: full CRISP-DA

Phần training còn lại:

- bật đủ ACCEPT/REVISE/SOFT/ABSTAIN;
- bật target boundary loss;
- bật progressive class-wise threshold controller;
- update Distribution Bank từ source + ACCEPT target, và có thể thêm high-confidence REVISE với trọng số nhỏ;
- duy trì temporal memory và EMA teacher.

14.5 Một iteration đầy đủ

1. Sample source batch (x_s, y_s) và target batch x_t.
2. Forward source qua student; tính $\mathcal{L}_{\text{sup}}$, $\mathcal{L}_{\text{bnd}}^{S}$ và source features.
3. Tạo x_t^{w1}, x_t^{w2}; forward qua EMA teacher.
4. Sinh raw pseudo-label $\tilde{y}_t$ và teacher dense feature F_t^T.
5. Tính q_i, a_i, d_i, s_i, b_i; tạo trust vector.
6. Forward trust MLP để lấy r_i.
7. Four-State Policy tạo $Y_t^*, Q_t^*, M_{hard}, M_{soft}, W_t$.
8. Tạo strong view x_t^s và transform supervision đúng coordinate.
9. Forward strong target qua student; tính $\mathcal{L}_{\text{hard}}, \mathcal{L}_{\text{soft}}, \mathcal{L}_{\text{cons}}, \mathcal{L}_{\text{bnd}}^{T}, \mathcal{L}_{\text{proto}}$.
10. Tạo corrupted source sample và tính $\mathcal{L}_{\text{trust}}$.
11. Tổng hợp $\mathcal{L}$; backprop và optimizer step cho θ, ψ, ϕ.
12. EMA update teacher.
13. Detached update Distribution Bank: source statistics luôn hợp lệ; sau warm-up thêm ACCEPT target và optional high-confidence REVISE với trọng số thấp hơn.
14. Update temporal memory.
15. Cuối epoch/window: update class-wise thresholds theo curriculum controller.

14.6 Thứ tự update trong một iteration để tránh leakage

Một iteration nên giữ thứ tự cố định. Trước hết Teacher forward trên weak target views và Student forward trên source; Bank được query bằng target feature nhưng chưa update bằng target của iteration hiện tại. Sau đó MTE sinh r_i, BPCC cung cấp thresholds, Four-State Policy tạo refined supervision. Student forward trên strong target view, tính loss và optimizer step. Tiếp theo cập nhật EMA Teacher. Cuối iteration, dùng detached source features và các target features đã được policy xác nhận để update Bank; cuối cùng cập nhật temporal memory và running class statistics. Thứ tự "query trước, update sau" tránh việc một target feature tự làm thay đổi bank rồi ngay lập tức được đánh giá bằng bank đã chứa chính nó.

1. Sample (x_s, y_s) và x_t; sinh $x_t^{w_1}, x_t^{w_2}, x_t^s$ cùng transform metadata.
2. Teacher forward weak views $\rightarrow p_t^T, F_t^T, B_t^T$; align weak predictions.
3. Student forward source $\rightarrow p_s^S, F_s^S, B_s^S$.
4. Query current bank bằng $F_t^T \rightarrow d_i, p_i^{proto}$; đọc temporal memory $\rightarrow s_i$.
5. MTE $\rightarrow r_i$; BPCC $\rightarrow \tau_{high,c}, \tau_{low,c}$; Four-State Policy $\rightarrow$ hard/soft targets + masks.
6. Warp refined targets sang strong-view coordinates; Student forward x_t^s.
7. Tính $\mathcal{L}_{total}$, backprop và optimizer step cho trainable modules.
8. EMA update Teacher.
9. Detached bank update bằng source + reliable target theo stage policy.
10. Update temporal memory, coverage/reliability running statistics và logs.

14.7 Warm-up và chuyển pha

Stage transition nên dựa trên training progress tối thiểu và diagnostic stability thay vì chỉ một epoch cố định. Ví dụ Bank chỉ nhận target khi source bank đã có đủ occupancy cho phần lớn classes và Teacher pseudo-label precision proxy/entropy đã ổn định. Full Four-State Policy chỉ bật khi MTE calibration AUROC/ECE trên corrupted-source validation đạt mức chấp nhận được. Cách này giảm rủi ro một module chưa ổn định tạo lỗi dây chuyền cho module sau.

14.8 Optimizer và state update không đồng tốc

Student optimizer có thể update mỗi minibatch, Teacher EMA cũng mỗi minibatch, nhưng BPCC thresholds và class statistics có thể update theo epoch hoặc sau một fixed window để giảm noise. Bank statistics có thể update mỗi iteration nếu batch đủ sample, nhưng rare-class mode nên dùng accumulated queue/window. Temporal memory update mỗi khi image ID xuất hiện. Việc các state có cadence khác nhau là bình thường; implementation cần tách rõ counters để reproducibility tốt.

15 Pseudo-code tham khảo

```
for each training iteration:
    (xs, ys) = sample_source_batch()
    xt       = sample_target_batch()

    # 1) Source branch
    Fs, ps = student(xs)
    bs     = boundary_head(Fs)
    L_sup  = CE(ps, ys)
    L_bndS = boundary_loss(bs, boundary_gt(ys))

    # 2) Teacher target predictions
    xw1, xw2 = weak_aug_1(xt), weak_aug_2(xt)
    with no_grad():
        Ft, pt1 = ema_teacher(xw1)
        _,  pt2 = ema_teacher(xw2)
        pt2 = warp_to_view1(pt2)
        yraw = argmax(pt1)

    # 3) Multi-evidence trust
    q = teacher_confidence(pt1)
    a = augmentation_consistency(pt1, pt2)
```

```
    d, pproto = prototype_bank.score(Ft, yraw)
    s = temporal_memory.stability(xt.ids, pt1)
    b = ema_boundary_head(Ft)
    r = trust_estimator(q, a, d, s, b, yraw)

    # 4) Four-state pseudo-label policy
    Yhard, Qsoft, Mhard, Msoft, weights = policy(
        r, pt1, pproto, class_thresholds, b, s, a)

    # 5) Strong student target branch
    xs_t, transformed_targets = strong_aug_and_warp(
        xt, Yhard, Qsoft, Mhard, Msoft)
    Ft_s, pt_s = student(xs_t)
    bt_s       = boundary_head(Ft_s)

    L_hard = masked_CE(pt_s, Yhard, Mhard, weights)
    L_soft = masked_KL(pt_s, Qsoft, Msoft)
    L_cons = reliability_weighted_KL(pt_s, warp(pt1), r)
    L_bndT = reliable_target_boundary_loss(bt_s, transformed_targets)
    L_proto = prototype_distance_loss(Ft_s, Yhard, Mhard)

    # 6) Trust calibration on corrupted source
    xsc = corrupt(xs)
    trust_target = correctness_label(student(xsc), ys)
    trust_evidence = build_source_evidence(xsc, ys)
    L_trust = weighted_BCE(trust_estimator(trust_evidence), trust_target)

    # 7) Optimize student / boundary / trust networks
    L = L_sup + ... + L_trust
    optimizer.zero_grad()
    L.backward()
    optimizer.step()

    # 8) State updates outside gradient
    ema_update(teacher, student)
    prototype_bank.update(source_features=Fs,
                          target_features=Ft,
                          accepted_mask=Mhard & is_ACCEPT)
    temporal_memory.update(xt.ids, pt1)

end epoch/window:
    update_classwise_thresholds(coverage, reliability)
```

16 Kiểm tra tính đúng đắn của luồng và các invariant triển khai

Trước khi chạy benchmark đầy đủ, nên kiểm tra một số invariant. Thứ nhất, mọi target evidence map phải cùng coordinate system và resolution. Thứ hai, tổng state masks tại mỗi pixel phải bằng một: $M_A + M_R + M_S + M_X = 1$. Thứ ba, p_i^{proto} và p_i^{soft} phải là normalized distributions. Thứ tư, bank parameters không xuất hiện trong optimizer parameter groups. Thứ năm, Teacher parameters không nhận gradient. Thứ sáu, target features dùng update Bank phải đến từ pixel đã được policy gate ở iteration trước hoặc cuối iteration hiện tại sau khi query xong.

Một bộ unit test tối thiểu nên kiểm tra: (i) identity augmentation cho $a_i \approx 1$; (ii) target feature đặt đúng bằng μ_{ck} cho compatibility cực cao; (iii) prototype posterior với two-top classes gần bằng nhau tạo margin nhỏ và không REVISE; (iv) khi r_i tăng qua τ_{low} và τ_{high}, state chuyển đúng ABSTAIN $\rightarrow$ SOFT $\rightarrow$ ACCEPT nếu không có conflict; (v) SOFT target sum bằng 1 trong tolerance số học; (vi) EMA Teacher và EMA Bank thay đổi chậm hơn Student/batch statistics.

17 Phân tích độ phức tạp và chi phí bộ nhớ

Phần inference của CRISP-DA không tăng đáng kể chi phí nếu chỉ giữ Student segmentation network. Chi phí bổ sung chủ yếu xuất hiện trong training. EMA Teacher yêu cầu một bản sao model weights và thêm weak-view forward. Prototype Bank với diagonal covariance có kích thước xấp xỉ $\sum_c K_c(2D+1)$ scalars; với $C = 19$, $K = 3$, $D = 256$, đây là rất nhỏ so với backbone. Temporal memory mới là thành phần có thể lớn nếu lưu full C-dimensional probability cho mọi target image. Vì vậy nên lưu low-resolution FP16 memory hoặc compressed top-k/EMA logits tùy GPU/RAM budget.

Trust Estimator dùng 1×1 Conv nên chi phí nhỏ so với encoder. Prototype query có complexity xấp xỉ $O(BH'W'\sum_c K_c D)$ nếu so với mọi class/mode; có thể vectorize hoàn toàn bằng tensor operations. Nếu chỉ cần compatibility với Teacher class, d_i rẻ hơn; full p_i^{proto} cho REVISE cần all-class score. Một optimization là tính full posterior chỉ cho pixel có r_i dưới ACCEPT threshold hoặc Teacher–Bank disagreement candidate, nhưng V1 nên ưu tiên code rõ ràng trước khi tối ưu.

18 Tensor I/O specification

Bảng 3: Input/output của từng thành phần.

Module	Input	Output	Gradient?
Student Encoder	$x \in \mathbb{R}^{B\times3\times H\times W}$	F_1, F_2, F_3, F_4; fused $F^{dense} \in \mathbb{R}^{B\times256\times H/4\times W/4}$	Có
Segmentation Head	F^{dense}	logits/probability $p \in \mathbb{R}^{B\times C\times H\times W}$	Có
Boundary Head	F^{dense}	$b \in [0,1]^{B\times H\times W}$	Có
EMA Teacher	x_t^w	p_t^T, F_t^T, b_t^T	Không
Prototype Bank	class features, labels/masks	$(\pi_{ck}, \mu_{ck}, \Sigma_{ck})$, d_i, p_i^{proto}	Không; EMA state
Aug. Consistency	$p^{T,1}, p^{T,2}$	$a_i \in [0,1]$	Không
Temporal Memory	current teacher probabilities + image ID	$s_i \in [0,1]$	Không; state update
Trust Estimator	$q_i, a_i, d_i, s_i, b_i, e_c, e_\rho$	$r_i \in [0,1]$	Có
Four-State Policy	$r_i, p_i^T, p_i^{proto}, \tau_{c,\rho}$	state; $Y_t^*, Q_t^*, M_{hard}, M_{soft}, W_t$	Không; rule/policy
Curriculum Controller	class-wise coverage/reliability	updated $\tau_{c,\rho}$	Không; controller

19 Cấu hình huấn luyện khuyến nghị để bắt đầu

Các giá trị dưới đây là điểm khởi đầu để tune, không phải hyperparameter đã được thực nghiệm xác nhận cho CRISP-DA.

- Backbone: MiT-B5 pretrained.
- Optimizer: AdamW.
- LR backbone: khoảng 6×10^{-5}; LR heads/trust: khoảng 6×10^{-4}.
- Weight decay: 0.01.
- EMA α: 0.999.
- Prototype EMA m: 0.99.
- Number of modes K_c: bắt đầu với 3.
- Trust MLP: 25–64–32–1, Dropout 0.1.
- Temporal memory EMA γ: 0.9.
- Initial τ_{high}: calibration theo target precision proxy, không nhất thiết cố định 0.95 cho mọi class.
- τ_{low}: thấp hơn τ_{high} khoảng 0.2–0.3 như điểm khởi đầu.
- Revision margin δ: tune trên source corruption validation.

20 Inference

Sau training, bỏ toàn bộ training-only modules:

- EMA Teacher;
- Boundary Head nếu chỉ cần semantic map;
- Multi-Prototype Bank;
- Trust Estimator;

- Four-State Policy;
- Temporal Memory;
- Curriculum Controller.

Inference chỉ còn:

$$\boxed{x_t \to E_\theta \to H_\theta \to p_t \to \hat{y}_t}.$$

Do đó inference latency gần bằng segmentation baseline.

21 Hướng dẫn vẽ sơ đồ kiến trúc trong diagrams.net

21.1 Bố cục đề xuất

Dùng landscape layout. Chia figure thành ba vùng chính:

1. **Source branch** ở trên-trái: source input → Student Encoder → Segmentation/Boundary/Class Feature outputs.
2. **Target trust/refinement branch** ở trên-phải: target → two weak views → EMA Teacher → evidence → Trust Estimator → Four-State Policy.
3. **Training/update branch** ở dưới: strong target view + refined supervision → Student → losses → optimizer → EMA/prototype/memory updates.

Inference đặt thành một box nhỏ tách biệt ở góc phải-dưới.

21.2 Mũi tên bắt buộc

Chỉ nên vẽ các mũi tên logic sau:

- source input → student encoder;
- student encoder → segmentation head, boundary head, source class features;
- source class features → prototype bank;
- target → weak views → EMA teacher;
- EMA teacher logits/features → evidence/trust;
- prototype bank → distribution evidence và REVISE evidence;
- temporal memory/boundary head → trust;
- trust → reliability → four-state policy;
- policy → refined supervision → strong student target branch;
- student outputs → losses → optimizer;
- optimizer → student update;
- student update ⇢ EMA teacher;
- reliable detached features ⇢ prototype bank;
- teacher probabilities ⇢ temporal memory.

21.3 Không nên vẽ

Không nối trực tiếp raw pseudo-label vào student trước Four-State Policy; không nối ABSTAIN vào hard/soft losses; không backprop arrow vào EMA teacher hoặc bank; không nối prototype bank trực tiếp vào segmentation logits nếu phiên bản triển khai không có classifier branch riêng.

22 Ablation bắt buộc

Để chứng minh core hypothesis, nên triển khai theo thứ tự:

1. EMA self-training baseline + global confidence threshold.
2. + class/role-conditioned trust estimator chỉ với confidence.
3. + augmentation consistency.
4. + temporal stability.
5. + multi-prototype distribution evidence.
6. Keep/Ignore → Four-State Policy.
7. + boundary-aware evidence/loss.
8. + Balanced Progressive Coverage Curriculum.

Các ablation quan trọng nhất:

$$\text{single prototype vs multi-prototype},$$

$$\text{confidence threshold vs calibrated multi-evidence trust},$$

$$\text{Keep/Ignore vs Accept/Revise/Soft/Abstain}.$$

23 Evaluation protocol

Nếu contribution nói về pseudo-label reliability, chỉ mIoU là không đủ. Nên báo:

- mIoU và per-class IoU;
- head-class vs tail-class IoU;
- Boundary-IoU hoặc trimap boundary IoU;
- pseudo-label precision, recall, coverage;
- precision–coverage curve;
- ECE, NLL hoặc Brier score cho calibration;
- correction precision: tỉ lệ REVISE thực sự sửa đúng;
- abstention utility: performance theo coverage khi loại uncertain pixels;
- training compute, memory và inference latency.

24 Research hypotheses

H1 – Reliability hypothesis

Class- and spatial-role-conditioned multi-evidence trust estimation dự đoán pseudo-label correctness tốt hơn raw softmax confidence dưới domain shift.

H2 – Revision hypothesis

Multi-prototype distribution evidence có thể sửa một phần confidently wrong target pseudo-label với correction precision đủ cao nếu dùng margin/safeguard và conservative warm-up.

H3 – Utilization hypothesis

Four-State Policy đạt precision–coverage trade-off tốt hơn Keep/Ignore bằng cách giữ hard supervision cho confident pixels, soft supervision cho ambiguous-but-useful pixels và abstain với conflict mạnh.

H4 – Balance hypothesis

Class-wise progressive coverage controller giúp tăng tail-class coverage mà không gây sụt đáng kể pseudo-label precision.

25 Giới hạn và rủi ro của kiến trúc đề xuất

25.1 Prototype poisoning

Nếu target pseudo-label ban đầu kém, target feature có thể contaminate bank. Giảm rủi ro bằng source bootstrap, ACCEPT-only bank update, EMA nhỏ và revision margin.

25.2 Source-corruption calibration gap

Correctness mapping học từ corrupted source không đảm bảo giống target thật. Đây là một research question cần ablation: source corruption types nào chuyển được sang target reliability tốt nhất?

25.3 Temporal memory cost

Lưu full $C \times H \times W$ probability history rất tốn RAM. Nên dùng low-resolution map, FP16, CPU storage hoặc top-1/confidence/entropy summary.

25.4 Threshold/controller sensitivity

Four-State Policy vẫn chứa thresholds. Mục tiêu không phải loại bỏ hoàn toàn hyperparameter mà thay global threshold bằng calibrated, class-aware và progressively controlled thresholds. Nên báo sensitivity.

25.5 Closed-set limitation

Phiên bản cơ sở chưa xử lý unknown classes. Với open-vocabulary UDA, prototype posterior và abstention cần thêm unknown modeling.

26 Phiên bản tối giản để triển khai V1

Nếu thời gian/compute hạn chế, nên giữ ba contribution chính:

1. Multi-Prototype Distribution Bank;
2. Multi-Evidence Trust Estimator với q, a, d, s;
3. Four-State Policy.

Tạm bỏ Boundary Head và Curriculum Controller. Sau khi chứng minh core gain bằng pseudo-label precision–coverage và mIoU, mới thêm boundary/curriculum như extension. Cách này làm ablation rõ và giảm khả năng "module stacking".

27 Checklist triển khai

1. Reproduce EMA self-training baseline ổn định.
2. Thêm dense feature projector và validate shape/gradient.
3. Bootstrap source prototype bank; visualize t-SNE/UMAP per class/mode.
4. Implement prototype posterior và kiểm tra class confusion trên source validation.
5. Implement weak-view alignment + consistency.
6. Implement temporal memory với image ID và geometric warp.
7. Train trust estimator trên source corruption; báo AUROC correctness/ECE.
8. Implement Four-State Policy; kiểm tra tỉ lệ mỗi state theo epoch/class.
9. Bật strong-view target training.
10. Sau khi ổn định, thêm boundary head và curriculum controller.

28 Kết luận

CRISP-DA (Calibrated Reliability-Informed Structured Pseudo-labeling for Domain Adaptation) được thiết kế như một framework **Advanced Self-Training & Pseudo-Label Refinement** cho UDASS. Contribution trung tâm không nằm ở backbone mới mà ở cách quản lý target supervision. Hệ thống tách rõ ba tầng: (i) **evidence acquisition** bằng teacher, augmentation, distribution, temporal và spatial cues; (ii) **reliability calibration** bằng Trust Estimator; (iii) **supervision action** bằng Four-State Policy và progressive class-wise curriculum. Nếu các ablation xác nhận giả thuyết, kiến trúc này có tiềm năng cải thiện không chỉ mIoU mà cả calibration, precision–coverage, boundary và tail-class performance trong self-training UDASS.

A Bảng cấu hình module chi tiết

Bảng 4: Module-level implementation specification.

Module	Layers / state	Output
MiT-B5 encoder	4 transformer stages; dims 64/128/320/512; depths 3/6/40/3	F_1, F_2, F_3, F_4
Dense projector	4 x 1×1 projection to 256; bilinear upsample; concat; 1×1 fusion 1024→256	F^{dense}
Segmentation head	3×3:256→256; Norm+GELU; Dropout; 1×1:256→ C	logits/probabilities
Boundary head	3×3:256→128; Norm+GELU; 3×3:128→64; Norm+GELU; 1×1:64→1; sigmoid	boundary map
EMA teacher	copy of encoder+seg head (+ boundary head optional), EMA state	weak-view target logits/features
Prototype bank	per class K_c tuples $(\pi, \mu, \Sigma_{diag})$	compatibility + class posterior
Temporal memory	per-image low-res EMA probability state	stability score s_i
Trust estimator	input 21-D (5 evidence + 16-D class embedding); FC64+GELU+Dropout; FC32+GELU; FC1+Sigmoid	r_i
Four-State policy	rule/controller using r, p^T, p^{proto}, τ	hard/soft/mask/weights
Coverage curriculum	EMA class statistics + threshold controller	$\tau_{c,\rho}(e)$

B Ký hiệu

Ký hiệu	Ý nghĩa
x_s, y_s	source image và source ground truth
x_t	target image không nhãn
E_θ, H_θ	student encoder và segmentation head
B_ψ	boundary head
$\bar{\theta}$	EMA teacher parameters
F^{dense}	fused dense embedding dùng cho prototype/trust
p^T, p^S	teacher và student probability distributions
$\tilde{y}$	raw teacher pseudo-label
$\mu_{ck}, \Sigma_{ck}, \pi_{ck}$	mean, diagonal covariance, mixture weight của mode k thuộc class c
q_i	teacher confidence
a_i	augmentation consistency
d_i	class-distribution compatibility
s_i	temporal stability
b_i	boundary probability / spatial-difficulty evidence
r_i	calibrated reliability
Y_t^*	refined hard target labels
P_t^{soft}	refined soft target distributions
M_{hard}, M_{soft}	hard/soft supervision masks
$\tau_{high,c}(t), \tau_{low,c}(t)$	class-specific dynamic thresholds từ BPCC

Tài liệu

[1] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," *NeurIPS*, 2021.

[2] S. Jeong, J. Kim, S. Kim, and D. Min, "Revisiting Domain-Adaptive Semantic Segmentation via Knowledge Distillation," *IEEE Transactions on Image Processing*, vol. 33, 2024.

[3] Q. Xu, Q. Wu, B. Jiang, J. Wang, Y. Chen, and J. Tang, "Unlocking Cross-Domain Synergies for Domain Adaptive Semantic Segmentation," *IEEE Transactions on Image Processing*, vol. 35, 2026.

[4] S. Wang, H. Chen, H. Huang, Y. He, D. Sun, X. Chen, X. Liu, Z. Wang, and K. Zhao, "Heuristic Self-Paced Learning for Domain Adaptive Semantic Segmentation under Adverse Conditions," 2026.

[5] A. Tarvainen and H. Valpola, "Mean Teachers Are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results," *NeurIPS*, 2017.

[6] Y. Zou, Z. Yu, B. V. K. Vijaya Kumar, and J. Wang, "Unsupervised Domain Adaptation for Semantic Segmentation via Class-Balanced Self-Training," *ECCV*, 2018.

[7] P. Zhang, B. Zhang, T. Zhang, D. Chen, Y. Wang, and F. Wen, "Prototypical Pseudo Label Denoising and Target Structure Learning for Domain Adaptive Semantic Segmentation," *CVPR*, 2021.

[8] L. Hoyer, D. Dai, H. Wang, and L. Van Gool, "MIC: Masked Image Consistency for Context-Enhanced Domain Adaptation," *CVPR*, 2023.

[9] N. Moradinasab, L. S. Shankman, R. A. Deaton, G. K. Owens, and D. E. Brown, "ProtoGMM: Multi-prototype Gaussian-Mixture-based Domain Adaptation Model for Semantic Segmentation," *arXiv:2406.19225*, 2024.